CAS III: Searching for Orbit Models*

Value Estimation on Subgroup Lattices

Romie Banerjee

October 2026

Abstract

In algorithmic statistics a model for a binary string x is a finite set containing it, and Kolmogorov's structure function records how well the simple models fit. In [1] a group acting on $X = \{0,1\}^n$ is read as a hypothesis about strings and the orbit of x as the model it assigns. This paper develops the search for such orbit models as a reinforcement-learning problem on subgroup lattices, for two ambient groups: all permutations of X, with groups given by Burnside-ring coordinates, and the linear group $\mathrm{GL}(n,2)$. The objective is the codelength of x under the coding scheme H, $\mathcal{L}_H(x) = \log \#\mathrm{orb}(H) + \log |H \cdot x|$, together with the codelength of H itself.

1 Introduction

A finite group acting on $X = \{0,1\}^n$ partitions it into orbits. The second paper of this series [1] treats such an orbit partition as a hypothesis about strings and the orbit of x as the model it assigns to x, and develops the resulting theory of symmetric structure functions for two ambient groups: the full symmetric group $\mathrm{Sym}(X)$, where every partition is symmetric and the theory reproduces Kolmogorov's structure function, and the linear group $\mathrm{GL}(n,2)$, where cells are the linearly homogeneous sets and the theory is genuinely restrictive. In [1, Section 9] every permutation group is given coordinates, a *type* and a *placement*, and the question is raised how to search them for good models.

This paper sets up that search as a reinforcement-learning problem on subgroup lattices. Given a string x, an agent moves through the lattice of subgroups of an ambient group G, down to maximal subgroups or sideways by cheap relabellings, looking for a subgroup H under which x has a short description. We call

$$\mathcal{L}_H(x) = \log \#\mathrm{orb}(H) + \log |H \cdot x|$$

the *codelength of x under the coding scheme H*: the index of the orbit of x followed by the position of x in it. The quantity to minimise is the total two-part codelength, the codelength of H itself plus $\mathcal{L}_H(x)$. Figure 1 shows the three pictures this involves: the walk in the lattice, the model it produces, and the model's place in the description profile of x.

Searching groups instead of models. The idea of the paper is to search for models of x not among all models but among subgroups, which are strong models that come with a map and coordinates. A model of x in the sense of Kolmogorov is any finite set $A \ni x$, and the models worth having are the strong ones, the cells of simple partitions [17]. Searched directly,

*CAS stands for Computational Algorithmic Statistics. This paper continues *CAS II: Orbits as Models* [1], which develops Kolmogorov's structure function under symmetry and supplies the background used here. Code: `https://github.com/romiebanerjee/CAS3`.

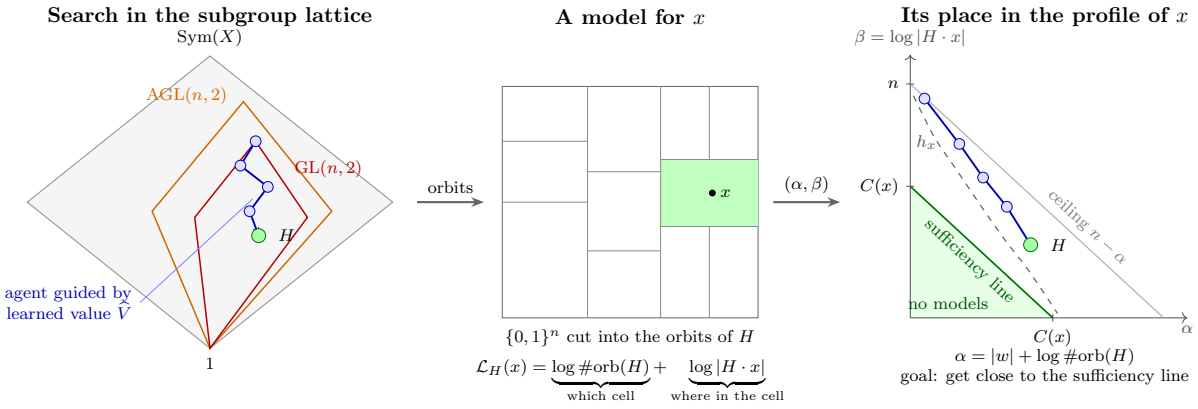


Figure 1: From a group to a model to a point of the profile. *Left:* the lattice of subgroups of $\mathrm{Sym}(X)$, $X = \{0,1\}^n$, which contains those of $\mathrm{AGL}(n,2)$ and $\mathrm{GL}(n,2)$. Given x, an agent starts at a seed, moves down to maximal subgroups or sideways by cheap relabellings, guided by a learned value function $\widehat{V}$, and stops at a subgroup H. *Middle:* the orbits of H partition X, and x is coded by the index of its orbit and its position in it, at codelength $\mathcal{L}_H(x)$. *Right:* each state w of the walk is a point (α, β), where $\alpha = |w| + \log \#\mathrm{orb}(H_w)$ pays for the hypothesis and the cell and $\beta = \log |H_w \cdot x|$ for the position in the cell. The walk moves toward the sufficiency line $\alpha + \beta = C(x)$ and stays above the profile h_x, which no search can compute. Along the lines of this paper, the walk can be restricted to groups of simple description, the strong sublattice of Section 6; then every model it reports is a strong model, and the collapse of Section 4, a descent to a group that names x, is ruled out by the budget.

this space is formless: there are 2^{N-1} sets $A \ni x$ and $2^{\Theta(N \log N)}$ partitions of X, with $N = 2^n$; nothing tells a strong model from a weak one; and evaluating a candidate means writing down sets of up to 2^n points. Subgroups change each of these three things.

- *Strong models by construction.* The orbit partition of H costs at most $C(H) + O(1)$, so every orbit of a cheap group is a strong model, and restricting to cheap groups is restricting to strong models. In $\mathrm{Sym}(X)$ nothing is lost: strong models are, up to $O(\varepsilon + \log n)$, exactly the orbits of cheap subgroups [1, Theorem 5.1].

- *A map.* The subgroups form a lattice whose local structure is known: the maximal subgroups of $\mathrm{Sym}(X)$ and of $\mathrm{GL}(n,2)$ are classified [14, 3, 12]. A down move to a maximal subgroup refines the partition, a sideways move by conjugation relabels it, and every move is a computable step with a code length. The map is what an agent walks on.

- *Coordinates.* Each group has a type and a placement (Section 2), and the codelength is read off the coordinates without touching the 2^n points. In $\mathrm{Sym}(X)$ the type is a Burnside-ring class and $\mathcal{L}_H(x)$ is a sum of indices and double-coset counts (Section 10.2). In $\mathrm{GL}(n,2)$ an element g fixes the subspace $\ker(g-1)$, so by Burnside's lemma $\#\mathrm{orb}(H) = |H|^{-1} \sum_{g \in H} 2^{\dim \ker(g-1)}$, and $|H \cdot x| = |H : \mathrm{Stab}_H(x)|$: both are computed from H by linear algebra over $\mathbb{F}_2$, without listing the points of X (Section 11.2).

The two ambient groups use these features differently. In $\mathrm{Sym}(X)$ there are more groups than partitions, since every partition is the orbit partition of its Young subgroup; the gain is not fewer candidates but structure. In $\mathrm{GL}(n,2)$ the map is also far smaller: about $2^{\Theta(n^4)}$ subgroups against $2^{\Theta(n2^n)}$ partitions, at the price of seeing only linearly homogeneous structure.

Why the search needs learning. Even the map is far too large to enumerate, or to walk through systematically.

- *All permutations.* Put $N = |X| = 2^n$. The $N/2$ disjoint transpositions $(y,\, y+e_1)$ generate an elementary abelian group of rank $N/2$, and its subspaces of dimension $N/4$ alone number at least $2^{N^2/16}$, since a Gaussian binomial $\binom{m}{k}_2$ is at least $2^{k(m-k)}$. Roney-Dougal and Tracey [24], settling a conjecture of Pyber [22], show that this is the true order of growth: $\log|\mathrm{Sub}(\mathrm{Sym}(X))| = N^2/16 + O(N^{3/2}) = 4^{n-2} + O(2^{3n/2})$.

- *Linear groups.* The unipotent radical of a maximal parabolic subgroup of $\mathrm{GL}(n,2)$, the matrices $\left(\begin{smallmatrix} I & A \\ 0 & I \end{smallmatrix}\right)$ with A of size $\lfloor n/2 \rfloor \times \lceil n/2 \rceil$, is elementary abelian of rank $m = \lfloor n/2 \rfloor \lceil n/2 \rceil$, so $\mathrm{GL}(n,2)$ has at least $2^{\lfloor m/2 \rfloor \lceil m/2 \rceil} \approx 2^{n^4/64}$ subgroups. Conversely, a group K is generated by at most $\log|K|$ elements, because each new generator at least doubles the subgroup generated so far; so a group G has at most $|G|^{\log|G|}$ subgroups, and $\mathrm{GL}(n,2)$ fewer than 2^{n^4}. Since $\mathrm{GL}(n,2) \le \mathrm{AGL}(n,2) \le \mathrm{GL}(n+1,2)$, the affine group lies in between.

Exact counts exist only for toy sizes. For $n = 3$, $\mathrm{GL}(3,2)$ has 179 subgroups and $\mathrm{Sym}(X) = S_8$ has 151,221; for $n = 4$, S_{16} already has about 5.3×10^{15} [20]. For comparison, Go has about $2.08 \times 10^{170} \approx 2^{566}$ legal positions [32].

ambient group	order	$\log_2$ of the number of subgroups	exceeds Go for
$\mathrm{Sym}(X)$, $N = 2^n$	$N!$	$N^2/16 + O(N^{3/2})$	$n \ge 7$
$\mathrm{AGL}(n,2)$	$2^n\lvert\mathrm{GL}(n,2)\rvert$	between those of $\mathrm{GL}(n,2)$ and $\mathrm{GL}(n+1,2)$	$n \ge 14$
$\mathrm{GL}(n,2)$	$< 2^{n^2}$	between $\approx n^4/64$ and n^4	$n \ge 14$

The lattices are also wide and deep. Below the root of $\mathrm{Sym}(X)$ there are at least $\binom{N}{N/2-1} \ge 2^N/(N+1)$ maximal subgroups (the intransitive $S_k \times S_{N-k}$ with $k = N/2 - 1$), and below $\mathrm{GL}(n,2)$ at least $2^{\lfloor n^2/4 \rfloor}$ (the stabilisers of $\lfloor n/2 \rfloor$-dimensional subspaces). The longest chain of subgroups of $\mathrm{Sym}(X)$ has length $3N/2 - 2$ [6], and that of $\mathrm{GL}(n,2)$ length less than n^2. A Go position offers at most 362 moves, and a game lasts a few hundred.

Brute force is therefore out, as it is in Go, and the response is the one of AlphaGo [28] and AlphaZero [29]: a learned value function evaluates positions that cannot be searched to the end, and a tree search, here *Monte Carlo lattice search*, spends computation where that function is uncertain. Closer in spirit is AlphaTensor [10], which recasts a search for algebraic objects, low-rank decompositions of the matrix multiplication tensor, as a single-player game. Three features set our game apart.

- It is single-player and deterministic, with dense rewards: every move is scored by the decrease in description length it achieves, and there is no opponent to model.

- Its branching is not merely large but exponential in $|X|$, and it becomes manageable only because moves are paid for. With prefix-free move codes, a budget of ε bits leaves at most $(\varepsilon+1)2^{\varepsilon}$ reachable groups (Theorem 6.8). Restricting the walk to groups of simple description is what turns the lattice into a game of bounded width, and it is also what makes every model found a strong one.

- A Go engine knows when it has won. A model searcher never knows how close it is: the target is the sufficiency line $\alpha + \beta = C(x)$, which is uncomputable. The value of a position can only be estimated by exploring from it, which is the task of the value function.

Summary of results. Sections 3 to 9 develop a theory that holds for every ambient group. By Burnside's lemma $\mathcal{L}_H(x) = n + \log(1+\varepsilon_H) - \log|\mathrm{Stab}_H(x)|$, so $\mathcal{L}$ is a negative log-likelihood and minimising it is maximum likelihood. Without a cost the problem collapses: the minimum of $\mathcal{L}$ below H is attained by $\mathrm{Stab}_H(x)$, and over the whole ambient group it equals the logarithm of the number of orbits of a point stabiliser (1 bit for $\mathrm{Sym}(X)$ and $\mathrm{AGL}(n,2)$, $\log 3$ for $\mathrm{GL}(n,2)$), by a hypothesis that names x. The cost is therefore the substance of the problem. Restricting

to cheap hypotheses is restricting to strong models; the subgroups of bounded description length have no exact lattice characterization, but a decidable inner one, the *strong sublattice* of bounded chain entropy, certifies them. With rewards equal to decreases of the code length and no discounting, model search is value estimation: the optimal value of a subgroup is the gap to the best model below it, it satisfies a Bellman recursion over maximal subgroups, it lies between 0 and the computable bound $\delta_H - \delta_{\mathrm{Stab}_H(x)}$. We give Monte Carlo exploring starts, *Monte Carlo lattice search* (MCTS adapted to the lattice), and a training loop that combines them. Because the lattice, the moves and their costs do not depend on x, one exploration serves every string: an explored graph gives certified lower bounds on the value for all strings at once, which parallelises the training.

Section 10 treats all permutations. In Burnside coordinates $\mathcal{L}$ is the logarithm of the number of summands plus the logarithm of the size of the summand containing x, the value bound is a ratio of double-coset counts, and the lattice below a hypothesis is the subgroup lattice of its abstract group, so every quantity the search needs is a table lookup. The search is complete for strong models but its information lies in the placement. Section 11 treats $\mathrm{GL}(n, 2)$, where hypotheses are linearly homogeneous and searchable but provably blind to some simple structure. Section 12 nests the two, $\mathrm{GL}(n, 2) \leq \mathrm{AGL}(n, 2) \leq \mathrm{Sym}(X)$, and explains how running both estimates how much of the structure of a string is linear-symmetric.

Organisation. Most of the theory does not depend on the ambient group, and we develop it once: the objective and its likelihood reading (Section 3), the collapse of the uncosted problem (Section 4), the cost of a hypothesis (Section 5), the strong sublattice (Section 6), the decision process (Section 7), value functions (Section 8) and algorithms (Section 9). Section 10 specialises to $\mathrm{Sym}(X)$ in Burnside coordinates, Section 11 to $\mathrm{GL}(n, 2)$, and Section 12 compares them and closes with an outlook.

Three findings.

1. *$\mathcal{L}$ is a likelihood.* By Burnside's lemma, $\mathcal{L}_H(x) = n + \log(1 + \varepsilon_H) - \log|\mathrm{Stab}_H(x)|$ with $\varepsilon_H = \sum_{u \neq 1} |\mathrm{Fix}(u)|/N$, and $\mathcal{L} = -\log p_H(x)$ for the distribution that picks a uniform orbit and then a uniform point in it (Theorem 3.4).

2. *Without a cost the problem collapses.* The minimum of $\mathcal{L}$ below H is attained by $\mathrm{Stab}_H(x)$, and over the whole ambient group by its point stabiliser, with value 1 bit in $\mathrm{Sym}(X)$ and $\mathrm{AGL}(n, 2)$ and $\log 3$ in $\mathrm{GL}(n, 2)$ (Theorem 4.1). The minimiser names x; it is the maximum-likelihood point mass. All learnable structure lies in the trade-off with the cost.

3. *Model search is value estimation.* With rewards equal to decreases of the code length and no discounting, the optimal value of a subgroup is the gap between its model and the best model below it. It obeys a Bellman recursion over maximal subgroups, and lies between computable bounds (Section 8).

The two ambient groups. In $\mathrm{Sym}(X)$ the search is complete: every strong model is reachable, and the costed search is a search for strong models of any kind. But by [1, Proposition 9.4] the information of a model lies almost entirely in its placement, so the group structure says little. In $\mathrm{GL}(n, 2)$ the placement costs at most n^2 bits and the type carries the structure, so the hypotheses are genuinely symmetric and the space is small enough to search, but the maximal gap theorem of [1] shows that such a space is blind to some simple structure. In Burnside coordinates (Section 10) every quantity the search needs, including $\mathcal{L}$, the value bound and the lattice below a hypothesis, reduces to tables of the abstract group; in $\mathrm{GL}(n, 2)$ (Section 11) the same quantities are read off from ranks and modules. Since $\mathrm{GL}(n, 2) \leq \mathrm{AGL}(n, 2) \leq \mathrm{Sym}(X)$, the linear search is the down-set of one seed of the permutation search, and running both estimates how much of the structure of x is linear-symmetric (Section 12).

Conventions. $X = \{0,1\}^n$, $N = 2^n$; logarithms are to base 2; C is plain Kolmogorov complexity. Groups act on X on the left. "Up to $O(\log n)$" means up to an additive term of that order.

2 Background from CAS II

Numbers in parentheses refer to [1].

2.1 Models and structure functions

For $x \in X$ the *structure function* is $h_x(\alpha) = \min\{\log|S| : x \in S,\ C(S) \le \alpha\}$ [34]. With $C(x) = \kappa$, $h_x(\alpha) \ge \kappa - \alpha$ up to $O(\log n)$ (Lemma 2.2): no model lies below the *sufficiency line*. A model $A \ni x$ is *ε-strong* if a total program of length ε computes A from x (Definition 2.3, [33]); by the partition lemma (Lemma 2.4, [17]) strong models are essentially cells of simple partitions. For a family P of partitions, a P-model is a pair (p, B) with $x \in B \in p$, and

$$h_x^P(\alpha) = \min\{\log|B| :\ C(p,B) \le \alpha\}, \qquad h_{x,\varepsilon}^P(\alpha) = \min\{\log|B| :\ C(p) \le \varepsilon,\ C(p,B) \le \alpha\}$$

(Definition 4.1), with $C(p,B) = C(p) + C(B \mid p)$ up to $O(\log n)$ and $h_x \le h_x^P \le h_{x,\varepsilon}^P$ (Proposition 4.2).

2.2 Symmetric partitions

For an ambient group G acting on X, $\mathrm{orb}_G(U)$ is the orbit partition of $U \le G$ and $\mathrm{stab}_G(p)$ the subgroup preserving every cell of p. Then $\mathrm{orb}_G(U) \le p \iff U \le \mathrm{stab}_G(p)$ (Proposition 3.1); the closed subgroups correspond to the symmetric partitions Par_G; and $C(\mathrm{stab}_G p) \le C(p) + O(1) \le C(U) + O(1)$ whenever $\mathrm{orb}_G(U) = p$ (Proposition 3.3). In group language $h_x^{L(G)}(\alpha)$ is the least $\log|U \cdot x|$ with $C(U, U\cdot x) \le \alpha$, and it equals h_x^G up to $O(1)$ (Theorem 4.4). A cell has size $|B| = |U|/|\mathrm{Stab}_U(x)|$ (Proposition 4.5).

2.3 Coordinates

A subgroup $U \le \mathrm{Sym}(X)$ is the image of a faithful action of an abstract group H. Fix representatives $K_1, \dots, K_r$ of the conjugacy classes of subgroups of H. The action has a class in the Burnside ring [30],

$$b = \sum_{i=1}^{r} a_i [H/K_i], \qquad a_i \ge 0, \qquad \sum_i a_i |H : K_i| = N,$$

faithful iff $\bigcap_{a_i>0} \mathrm{core}_H(K_i) = 1$, and determined by its marks $\varphi_K(b) = |X_b^K|$. The *canonical realisation* U_b lays out a_i copies of H/K_i as consecutive intervals of X (Definition 9.1). Every $U \cong H$ is $\Phi_H(b,\sigma) = \sigma U_b \sigma^{-1}$ for a type b and a placement $\sigma \in \mathrm{Sym}(X)$, with $\Phi_H(b,\sigma) = \Phi_H(b,\sigma')$ iff $\sigma^{-1}\sigma' \in N(U_b)$ (Theorem 9.2), and up to $O(\log C(U))$

$$C(U) = C(H,b) + \min\{C(\sigma' \mid H, b) : \Phi_H(b,\sigma') = U\}$$

(Proposition 9.3). The orbits of $\Phi_H(b,\sigma)$ are σ applied to the intervals of the layout. Two moves navigate coordinates: *relabelling* $(b,\sigma) \mapsto (b, \pi\sigma)$, and *restriction* to $L \le H$, which refines each cell by the Mackey formula

$$\mathrm{Res}_L^H [H/K] = \sum_{LhK \in L\backslash H/K} \big[L/(L \cap hKh^{-1})\big]. \tag{1}$$

For $\mathrm{GL}(n,2)$ the same holds with $B(H)$ replaced by faithful $\mathbb{F}_2 H$-modules of dimension n and $\mathrm{Sym}(X)$ by $\mathrm{GL}(n,2)$; the placement then costs at most n^2 bits (Proposition 9.6).

2.4 The two ambient groups

All permutations. Every partition is Sym-symmetric, so $h_x^{\mathrm{Sym}} = h_x$ and $h_{x,\varepsilon}^{\mathrm{Sym}} = h_{x,\varepsilon}^{\mathrm{str}}$ up to $O(\varepsilon + \log n)$ (Theorem 5.1). But the canonical certificates are Young subgroups, whose types record only cell sizes, and the placement carries essentially all the information of a model (Proposition 9.4).

Linear symmetry. For $\mathrm{GL}(n, 2)$ and $x \neq 0$ the cells are exactly $\{0\}$ and the linearly homogeneous sets (Theorem 6.3), affine subspaces are cells of cheap hypotheses (6.1), and h_x^{GL} lies in the band $C(x) - \alpha \leq h_x^{\mathrm{GL}}(\alpha) \leq n - \alpha$ (Theorem 7.1), with both edges attained: by strings whose best models are affine (Corollary 8.1) and by stochastic, normal strings invisible to linear symmetry (Theorem 8.2, Corollary 8.3). Whether h_x^{GL} obeys the halving law is open (Question 7.2).

Search. Section 9.3 concludes that coordinates on $\mathrm{Sym}(X)$ are complete but uninformative, and coordinates on $\mathrm{GL}(n, 2)$ are searchable but blind: "a coordinate system in which the space of symmetry hypotheses is small enough to search is small enough to miss simple structure". A search should report how much of the structure of x is symmetric, by the deficiency of its best model against a compressor.

3 The objective

In Sections 3 to 9, $G \leq \mathrm{Sym}(X)$ is an ambient group and $H \leq G$. Write $\#\mathrm{orb}(H)$ for the number of orbits of H on X, $H \cdot x$ for the orbit of x, $\mathrm{Stab}_H(x)$ for its stabiliser and $\mathrm{Fix}(u) = \{y : uy = y\}$.

Definition 3.1 (Codelength under a hypothesis)**.** The *codelength of x under the coding scheme H* is

$$\mathcal{L}_H(x) = \log \#\mathrm{orb}(H) + \log |H \cdot x|.$$

A subgroup H determines a code for the points of X: list the orbits of H in a canonical order, send the index of the orbit of x, then the position of x in its orbit in lexicographic order. The code is fixed once H is known, and $\mathcal{L}_H(x)$ is its length. Strictly, the codeword has $\lceil \log \#\mathrm{orb}(H) \rceil + \lceil \log |H \cdot x| \rceil$ bits, at most two more than $\mathcal{L}_H(x)$; as usual in minimum description length we work with the idealised codelength and drop the ceilings [11]. In the three-part description of a symmetric model, $\mathcal{L}_H(x)$ is the term $C(B \mid p) + \log |B|$ with the cell named by a uniform index; in MDL terms it is the data-given-hypothesis part $L(x \mid H)$ of a two-part code. The other part, the *codelength of the hypothesis H* itself, is the subject of Section 5; the search minimises the total, the codelength of the hypothesis plus the codelength of x under it.

3.1 Invariance under closure

Lemma 3.2. *$\mathcal{L}_H(x)$ depends on H only through* $\mathrm{orb}(H)$. *Writing* $\mathrm{cl}(H) = \mathrm{stab}_G\mathrm{orb}(H)$, *$\mathcal{L}_{\mathrm{cl}H}(x) = \mathcal{L}_H(x)$, and if H is closed and $K \leq H$ then* $\mathrm{cl}(K) \leq H$.

Proof. Both terms are read off $\mathrm{orb}(H)$, and $\mathrm{orb}(\mathrm{cl}H) = \mathrm{orb}(H)$ [1, Proposition 3.1]. If $K \leq H$ then $\mathrm{orb}(K)$ refines $\mathrm{orb}(H)$, so a map preserving every cell of $\mathrm{orb}(K)$ preserves every cell of $\mathrm{orb}(H)$; hence $\mathrm{cl}(K) \leq \mathrm{cl}(H) = H$. □

The search may therefore be run on closed subgroups. Whether to close after each move is a computational and a modelling choice: in $\mathrm{Sym}(X)$ the closure is a Young subgroup and forgets the structure of the group that produced the partition [1, Example 9.5]; we return to this in Section 10.4.

3.2 The score identity

Definition 3.3 (Excess)**.** For $U \leq \mathrm{Sym}(X)$ put

$$\varepsilon_U = \frac{1}{N} \sum_{u \in U \setminus \{1\}} |\mathrm{Fix}(u)|, \qquad \delta_U = \log(1 + \varepsilon_U).$$

For $U \leq \mathrm{GL}(n, 2)$ acting on $\mathbb{F}_2^n$, $|\mathrm{Fix}(u)| = 2^{\,n - \mathrm{rank}(u-I)}$, so $\varepsilon_U = \sum_{u \neq 1} 2^{-\,\mathrm{rank}(u-I)}$.

Proposition 3.4 (Score identity)**.** *For every $H \leq \mathrm{Sym}(X)$ and $x \in X$,*

$$\#\mathrm{orb}(H) = \frac{N}{|H|}\,(1 + \varepsilon_H), \qquad \mathcal{L}_H(x) = n + \delta_H - \log |\mathrm{Stab}_H(x)|.$$

Proof. Burnside's lemma: $\#\mathrm{orb}(H) = \frac{1}{|H|} \sum_{u \in H} |\mathrm{Fix}(u)|$; the identity contributes N, and the rest is $N\varepsilon_H$. *Orbit–stabiliser:* $|H \cdot x| = |H|/|\mathrm{Stab}_H(x)|$. Adding the logarithms, $\log |H|$ cancels. □

Lemma 3.5 (Properties of the excess)**.** *(i) $\varepsilon_U \geq 0$, with equality iff $U = 1$. (ii) $U' \leq U$ implies $\varepsilon_{U'} \leq \varepsilon_U$ and $\delta_{U'} \leq \delta_U$. (iii) $1 + \varepsilon_U = \mathbb{E}_y |\mathrm{Stab}_U(y)|$ for y uniform on X. (iv) $\delta_U = \log(\#\mathrm{orb}(U)\,|U|/N)$.*

Proof. (i), (ii): ε_U is a sum of non-negative terms over $U \setminus \{1\}$. (iii): $\mathbb{E}_y |\mathrm{Stab}_U(y)| = \sum_{u \in U} \Pr_y[uy = y] = \sum_{u \in U} |\mathrm{Fix}(u)|/N$. (iv) is Theorem 3.4. □

Corollary 3.6 (Two readings of $\mathcal{L}$)**.** *(i)* Stabiliser surprisal:

$$\mathcal{L}_H(x) = n - \log \frac{|\mathrm{Stab}_H(x)|}{\mathbb{E}_y |\mathrm{Stab}_H(y)|};$$

$\mathcal{L} < n$ exactly when x has a larger stabiliser than an average point.

(ii) Likelihood: *let p_H choose an orbit of H uniformly and then a point of it uniformly. Then $p_H(x) = (\#\mathrm{orb}(H)\,|H \cdot x|)^{-1}$ and $\mathcal{L}_H(x) = -\log p_H(x)$. Minimising $\mathcal{L}$ is maximum likelihood over $\{p_H\}$.*

Proof. (i) combines Theorems 3.4 and 3.5(iii). (ii): summing p_H over an orbit gives $1/\#\mathrm{orb}(H)$, and over all orbits 1. □

Maximum likelihood without a model cost overfits; adding $C(H)$ turns the problem into two-part minimum description length [23, 11].

3.3 Random strings

Proposition 3.7 (Random points carry no signal)**.** *For y uniform on X, $\Pr[\mathrm{Stab}_H(y) \neq 1] \leq \varepsilon_H$, and with probability at least $1 - \varepsilon_H$, $\mathcal{L}_H(y) = n + \delta_H$ independently of y.*

Proof. A union bound with $\Pr[uy = y] = |\mathrm{Fix}(u)|/N$, and Theorem 3.4. □

Along a search, $\mathcal{L}$ depends on y only through its stabiliser. Replacing the logarithms by integer parts creates a spurious signal: for $\mathrm{Stab}_H(y) = 1$ and $\delta_H < 1$ the rounded score is n when $|H|$ is a power of 2 and $n - 1$ otherwise. Exact logarithms are used throughout.

4 The collapse of the uncosted problem

4.1 The theorem

Theorem 4.1 (Collapse)**.** *For every $H \leq \mathrm{Sym}(X)$ and $x \in X$,*

$$\min_{K \leq H} \mathcal{L}_K(x) = \mathcal{L}_{\mathrm{Stab}_H(x)}(x) = \log \#\mathrm{orb}\big(\mathrm{Stab}_H(x)\big).$$

Write $S_K = \mathrm{Stab}_K(x)$. The proof uses three facts: (A) δ is monotone (Theorem 3.5(ii)); (B) if L fixes x then $\mathcal{L}_L(x) = \log \#\mathrm{orb}(L)$; (C) if $L \leq L'$ then $\#\mathrm{orb}(L) \geq \#\mathrm{orb}(L')$, since every L'-orbit is a union of L-orbits.

Proof. Step 1. $\mathrm{Stab}_{S_K}(x) = S_K$, so by Theorem 3.4 $\mathcal{L}_K(x) = n + \delta_K - \log|S_K|$ and $\mathcal{L}_{S_K}(x) = n + \delta_{S_K} - \log|S_K|$; hence $\mathcal{L}_{S_K}(x) - \mathcal{L}_K(x) = \delta_{S_K} - \delta_K \leq 0$ by (A). *Step 2.* If $K \leq H$ then $S_K \leq S_H$, and by (B), (C), $\mathcal{L}_{S_K}(x) = \log \#\mathrm{orb}(S_K) \geq \log \#\mathrm{orb}(S_H) = \mathcal{L}_{S_H}(x)$. *Combination.* $\mathcal{L}_K(x) \geq \mathcal{L}_{S_K}(x) \geq \mathcal{L}_{S_H}(x)$ for all $K \leq H$, and $S_H \leq H$. □

Corollary 4.2 (Uncosted value)**.** $\mathcal{L}_H(x) - \min_{K \leq H} \mathcal{L}_K(x) = \delta_H - \delta_{\mathrm{Stab}_H(x)} \geq 0$.

Proof. Subtract the two formulas of Step 1 with $K = H$. □

Corollary 4.3 (Top values)**.** *Over all subgroups of an ambient group G, $\min_{K \leq G} \mathcal{L}_K(x) = \log \#\mathrm{orb}(G_x)$, the logarithm of the number of orbits of the point stabiliser $G_x = \mathrm{Stab}_G(x)$. In particular:*

Ambient group	*orbits of G_x*	$\min_K \mathcal{L}_K(x)$
$\mathrm{Sym}(X)$, $\mathrm{Alt}(X)$ *($N \geq 4$),* $\mathrm{AGL}(n,2)$*: 2-transitive*	$\{x\}$, $X \setminus \{x\}$	1
$\mathrm{GL}(n,2)$, $n \geq 2$, $x \neq 0$	$\{0\}$, $\{x\}$, $\mathbb{F}_2^n \setminus \{0,x\}$	$\log 3 \approx 1.58$

Proof. The first claim is Theorem 4.1 with $H = G$. A 2-transitive group has a point stabiliser transitive on the remaining points. $\mathrm{Sym}(X)$ and $\mathrm{AGL}(n,2)$ are 2-transitive, and so is $\mathrm{Alt}(X)$ for $N \geq 4$. For $\mathrm{GL}(n,2)$ and $v, w \notin \{0,x\}$, the sets $\{x,v\}$ and $\{x,w\}$ are linearly independent (distinct non-zero vectors over $\mathbb{F}_2$), so they extend to bases, and the linear map between the bases fixes x and sends v to w. □

Remark 4.4. The minimum is over $K \leq H$; over $K < H$ the only difference arises when H already fixes x.

4.2 Discussion: the collapse is overfitting

The stabiliser names x. Let G be computable from n and $S = G_x$. Then S is computable from x, so $C(S) \leq C(x) + O(\log n)$. Conversely $x \in \mathrm{Fix}(S)$, so $C(x) \leq C(S) + \log|\mathrm{Fix}(S)| + O(1)$. For the groups of Theorem 4.3, $\mathrm{Fix}(S) = \{x\}$ or $\{0,x\}$, hence

$$C(G_x) = C(x) \pm O(\log n).$$

The optimal "hypothesis" is not a regularity of x; it is x in another alphabet.

Example 4.5 ($n = 3$, all permutations)**.** $\mathrm{Stab}_{\mathrm{Sym}(X)}(x) = \mathrm{Sym}(X \setminus \{x\})$ has two orbits, so $\mathcal{L} = 1$ bit against 3 bits for writing x. But there are 8 such stabilisers, one per point, and naming the right one costs $\log 8 = 3$ bits; the total of 4 bits exceeds writing x down. The same computation for $\mathrm{GL}(3,2)$ (Theorem 11.1) gives $\log 3 + \log 7 = \log 21$ bits.

Below any H. By the sufficiency floor (Theorem 5.3), $C(K) + \mathcal{L}_K(x) \geq C(x) - O(\log n)$ for every K: every bit of $\mathcal{L}$ saved reappears in the cost of the hypothesis. $\mathrm{Stab}_H(x)$ saves the most, and indeed $C(\mathrm{Stab}_H(x) \mid H) \geq C(x \mid H) - \mathcal{L}_{\mathrm{Stab}_H(x)}(x) - O(\log n)$.

The maximum-likelihood point mass. p_{G_x} puts mass 1/2 on x in $\mathrm{Sym}(X)$ (and 1/3 in $\mathrm{GL}(n,2)$), the largest value of $p_H(x)$ in the family by Theorem 4.1: the closest the family comes to the point mass δ_x. This is maximum likelihood choosing the model after seeing the data; two-part MDL repairs it by charging $C(x)$ for the memorising hypothesis.

Consequences for search. (1) An uncosted score pulls every policy towards stabilisers of x, which look like improvements from n bits to one or two bits but explain nothing. (2) The uncosted lattice search needs no learning: one down-chain to $\mathrm{Stab}_H(x)$ is optimal and its value is known in closed form (Theorem 4.2). (3) All learnable structure lives in the trade-off between $\mathcal{L}$ and the cost of the hypothesis.

4.3 Cell codes

The uniform orbit index in $\mathcal{L}$ is a valid but conservative code for the cell: it charges special cells as if they were typical. Two refinements correspond to other likelihood families: a *fixed-set code*, which for $x \in \mathrm{Fix}(H)$ names x by its index in $\mathrm{Fix}(H)$ at cost $\log|\mathrm{Fix}(H)| \le \log \#\mathrm{orb}(H)$ (every fixed point is its own orbit); and a *rank code*, which names cells by position in a canonical order with a variable-length integer code. We keep $\mathcal{L}$ as the primary objective. The collapse persists under these codes, since $|\mathrm{Fix}(G_x)| \le 2$. Examples of the difference are in Theorems 10.7 and 11.2.

5 The cost of a hypothesis

5.1 Why the cost cannot be dropped

Minimising $\mathcal{L}$ alone rewards hypotheses that encode x (Section 4). What measures how well a symmetric hypothesis explains x is the full two-part code length $C(H)+\mathcal{L}_H(x)$ and its distance from $C(x)$. The working assumption that hypotheses are cheap must therefore be *enforced* by the space the search explores, and the search must be able to certify it.

5.2 Description schemes

Definition 5.1 (Description scheme)**.** A *description scheme* is a computable partial map that, given n, sends words w of a prefix-free set $\mathcal{W}_n \subseteq \{0,1\}^*$ to generating sets of subgroups $H_w \le G$. A state that records its word has cost $|w|$.

Lemma 5.2 (Certified cost)**.** *$C(H_w \mid n) \le |w| + O(1)$ and $C(\mathrm{orb}(H_w) \mid n) \le |w| + O(1)$.*

Proof. A fixed program decodes w, computes generators and from them the orbits. □

Proposition 5.3 (Sufficiency floor)**.** *For every word w and $x \in X$, $C(x) \le |w| + \mathcal{L}_{H_w}(x) + O(\log n)$. The point $(\alpha,\beta) = (|w| + \lceil \log \#\mathrm{orb}(H_w) \rceil,\ \log|H_w \cdot x|)$ satisfies $h^G_x(\alpha + O(1)) \le \beta$, and if $|w| \le \varepsilon$ also $h^G_{x,\varepsilon+O(1)}(\alpha + O(1)) \le \beta$; and $\alpha + \beta \ge C(x) - O(\log n)$.*

Proof. Describe x by w, the index of its orbit and its index in the orbit. The pair $(\mathrm{orb}(H_w), H_w \cdot x)$ is a Par_G-model with $C(p,B) \le \alpha + O(1)$ and $C(p) \le |w| + O(1)$. □

The search operates on *descriptions*, not on bare groups: every state carries a word, and every model it reports is a certified upper bound on the profile that cannot cross the sufficiency line.

Explicit generators are never compressive. A generator of a subgroup of $\mathrm{Sym}(X)$ written as a permutation costs about $\log N! \approx n2^n$ bits; a generator matrix of a subgroup of $\mathrm{GL}(n,2)$ costs n^2 bits. Both exceed the n bits of the string. A hypothesis that compresses x must have a description far shorter than a single explicit generator, so the effective search space is the set of subgroups with short descriptions in a chosen scheme, a vanishing fraction of the lattice. Three kinds of scheme are natural.

(a) Path codes from the root. The root G costs $O(1)$. Fix deterministic procedures listing, for $U \leq G$, representatives $M_1, \ldots, M_{r(U)}$ of the conjugacy classes of maximal subgroups and an ordered transversal of $N_U(M_i)$ in U. A down move to a conjugate of M_i is coded with $\log r(U) + \log |U : N_U(M_i)|$ bits. The code is uniform over conjugates, which is honest but expensive (Section 6).

(b) Structured languages. A grammar over named constructions with cheap arguments: natural actions on the cube, coordinate subspaces and intervals, block decompositions, groups generated by short words in cheap permutations. The language fixes what counts as simple; it encodes a prior over symmetric structure. Sections 10.4 and 11.3 give concrete languages.

(c) Learned priors. Let $q(a \mid w)$ be a policy over moves that *does not see* x. Arithmetic coding with q gives move sequences length $c_q(w) = -\sum_t \log q(a_t \mid w_{<t}) + O(1)$. Training q on a distribution of strings to minimise $\mathbb{E}_x \min_w [c_q(w) + \mathcal{L}_{H_w}(x)]$ is *amortised MDL*: it learns which symmetric structures are common. The code is valid on strings not used to train q.

5.3 Proposals versus codes

Lemma 5.4 (Proposal–code separation)**.** *Let the search choose moves by any procedure that may read x, and charge each chosen move its length under a code that depends only on the current word and n. Then Theorem 5.3 holds at every state. If the charge may depend on x, it can fail.*

Proof. The decoder reads moves one at a time, and each move's code depends only on the moves already decoded. For the converse, charge 1 bit for the move sequence leading to G_x when the data is x: the total $1 + \mathcal{L}_{G_x}(x)$ is below $C(x)$ for most x. □

So candidate moves may and should be generated from x; the charge for a chosen move must not depend on it. This matters at scale, since random candidates almost never relate to x.

5.4 Including the cost

Definition 5.5 (Potential)**.** For $\lambda \geq 0$ and a state with word w,

$$\Phi_\lambda(w, x) = \lambda |w| + \mathcal{L}_{H_w}(x),$$

λ times the codelength of the hypothesis plus the codelength of x under it. For $\lambda = 1$ this is the total two-part codelength.

The cost can enter in three ways.

1. *Lagrangian:* minimise Φ_λ. For $\lambda = 1$ this is the full two-part code; sweeping λ traces the lower convex hull of the achievable profile (Theorem 8.7).
2. *Budget:* minimise $\mathcal{L}$ subject to $|w| \leq \varepsilon$. This is the search version of $h^G_{x,\varepsilon}$, and exactly the restriction to strong models: every cell of a hypothesis within the budget is $(\varepsilon + O(1))$-strong (Theorem 6.2). The budget is the mechanism that enforces the restriction.
3. *Both* (recommended): a hard budget defines the class, a small $\lambda > 0$ breaks ties towards cheaper descriptions, and budgets are swept.

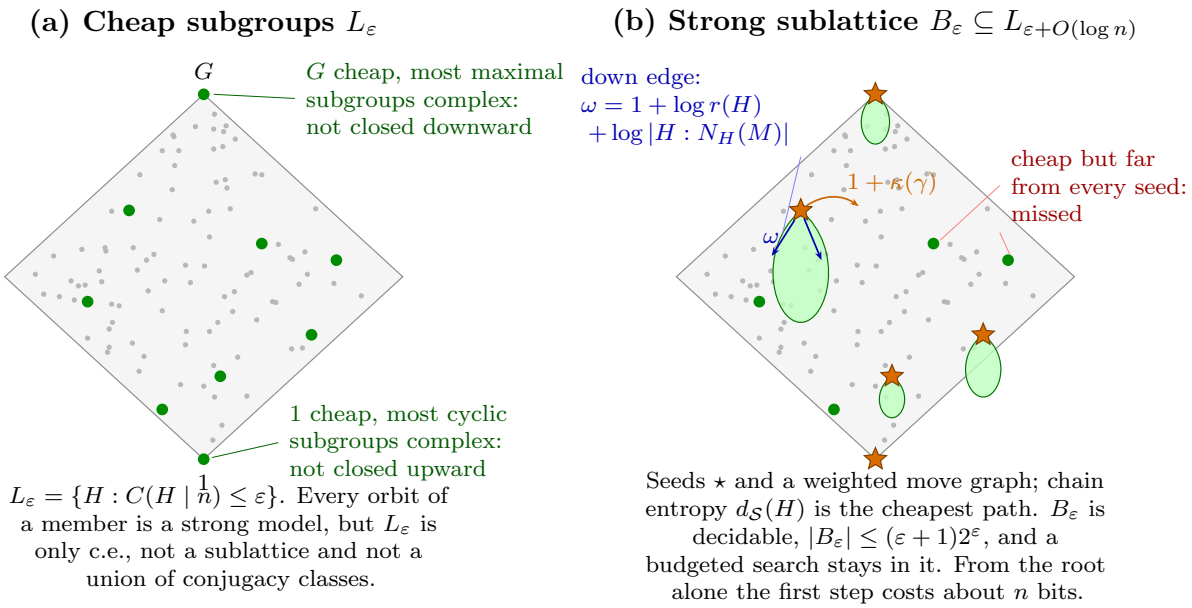


Figure 2: The strong sublattice. *(a)* The cheap subgroups L_ε (green) are few and scattered at every height of the lattice: the root is cheap but most of its maximal subgroups are not, and the trivial group is cheap but most cyclic subgroups are not (Theorem 6.3); every orbit of a cheap group is a strong model (Theorem 6.2). *(b)* Chain entropy measures the cheapest weighted path from a seed ($\star$) through down edges of weight ω and conjugation edges of weight $1 + \kappa(\gamma)$ (Theorem 6.7); the strong sublattice B_ε is the union of the balls below the seeds. It is decidable, small and sound (Theorem 6.8), the budgeted search lives in it (Theorem 6.9), and it misses cheap groups far from every seed.

5.5 Cost, placement and symmetry

$\mathcal{L}$ is equivariant under the ambient group, $\mathcal{L}_{H^g}(gx) = \mathcal{L}_H(x)$ with $H^g = gHg^{-1}$, but costs are not: $C(H^g) \le C(H) + C(g) + O(1)$. In coordinates [1, Proposition 9.3] the cost splits into type and placement. The two ambient groups differ sharply here: in $\mathrm{Sym}(X)$ a placement can carry up to about $n2^n$ bits and, for typical sets, carries almost all of the information of a model [1, Proposition 9.4]; in $\mathrm{GL}(n, 2)$ a placement carries at most n^2 bits [1, Proposition 9.6]. In both, compressive hypotheses cost less than n bits, so the only usable placements are those with short descriptions. Equivariance of the full objective holds exactly for the subgroup $\Gamma \le G$ of placements that preserve code lengths, and only that subgroup may be quotiented out of the value function (Section 8.4).

6 The strong sublattice

The motivation for restricting to cheap hypotheses is to restrict to *strong* models. This section describes the cheap subgroups in lattice terms, shows that they have no exact lattice characterization, and gives a decidable inner one; Fig. 2 summarises it.

6.1 Cheap subgroups and strong models

Definition 6.1. $L_\varepsilon = \{H \le G : C(H \mid n) \le \varepsilon\}$.

Proposition 6.2 (Cheap subgroups give strong models)**.** *If $H \in L_\varepsilon$, every cell of* $\mathrm{orb}(H)$ *is an $(\varepsilon + O(1))$-strong model of each of its points, and the closed members of L_ε correspond, up to $O(1)$ in ε, to the partitions $p \in \mathrm{Par}_G$ with $C(p) \le \varepsilon$.*

Proof. Given H, the map $x \mapsto H \cdot x$ is computed by a total program [1, Proposition 4.2(ii)]; for closed H, $C(H) = C(\mathrm{orb}H) \pm O(1)$ [1, Proposition 3.3]. □

By the partition lemma, restricting the search to strong symmetric models is restricting it to the closed subgroups in L_ε, and the profile it explores is $h^G_{x,\varepsilon}$.

6.2 No exact lattice characterization

Proposition 6.3. *Let $G \in \{\mathrm{GL}(n,2), \mathrm{Sym}(X)\}$.*

(i) Membership in L_ε is not decidable uniformly in n and ε; L_ε is only computably enumerable.

(ii) L_ε is not closed downwards: $G \in L_{O(\log n)}$, but for even n it has at least $2^{n^4/64-1}$ subgroups, so all but a 2^{-c} fraction have complexity at least $n^4/64 - c - 1$.

(iii) L_ε is not closed upwards: the trivial group is cheap, but G has at least 2^{n^2-n-2} cyclic subgroups, most of complexity close to n^2 or more.

(iv) L_ε is not closed under conjugation (Theorem 6.4).

Proof. (i) If membership were decidable uniformly, $C(H \mid n)$ would be computable on the infinite decidable set of all subgroups, and "the first subgroup of complexity at least m" would have complexity $O(\log m)$, a contradiction for large m. (ii) For $n = 2r$ the matrices $\left(\begin{smallmatrix} I & A \\ 0 & I \end{smallmatrix}\right)$ form a group $\cong \mathbb{F}_2^m$, $m = r^2$, with at least $2^{\lfloor m/2 \rfloor \lceil m/2 \rceil} \geq 2^{n^4/64-1}$ subgroups; they lie in $\mathrm{GL}(n,2) \leq \mathrm{Sym}(X)$. Fewer than 2^k objects have complexity below k. (iii) $|\mathrm{GL}(n,2)| \geq 2^{n^2-2}$ and every element has order less than 2^n. □

L_ε is neither an ideal, nor a filter, nor a sublattice, nor a union of conjugacy classes. What can be described is how it meets these structures.

6.3 How cheap subgroups sit in the lattice

Only hypotheses of complexity at most polynomial in n matter for compressing strings of length n, and for those the logarithmic error terms of symmetry of information are $O(\log n)$. All statements below are for such hypotheses.

Proposition 6.4 (Conjugacy classes)**.** *Let b be a type (a Burnside class for $\mathrm{Sym}(X)$, a module for $\mathrm{GL}(n,2)$) with canonical realisation U_b. For conjugates of complexity at most $\mathrm{poly}(n)$, up to $O(\log n)$,*

$$C(U_b^g) = C(b) + C\big(gN_G(U_b) \mid b\big).$$

So L_ε meets the class of U_b in a Kolmogorov ball of radius $\varepsilon - C(b)$ in the space of placements $G/N_G(U_b)$, of size at most $2^{\varepsilon - C(b) + O(\log n)}$ out of $|G : N_G(U_b)|$.

Proof. The type is computable from any realisation [1, Proposition 9.3], so U_b is computable from U_b^g and $C(U_b^g) = C(U_b) + C(U_b^g \mid U_b) \pm O(\log n)$ by symmetry of information; given U_b, the conjugate and the coset $gN_G(U_b)$ determine each other; and $C(U_b) = C(b) \pm O(1)$. □

Proposition 6.5 (Complexity along covering edges)**.** *Let $m(H)$ be the number of maximal subgroups of H and $u(M)$ the number of subgroups of G in which M is maximal. If M is maximal in H then, up to $O(\log n)$, $C(M) \leq C(H) + \log m(H)$ and $C(H) \leq C(M) + \log u(M)$.*

Proof. Given H, its maximal subgroups can be listed canonically; given M, so can the subgroups covering it. □

Proposition 6.6 (Definable operations)**.** *Up to $O(\log n)$, $C(H \wedge K)$ and $C(H \vee K)$ are at most $C(H) + C(K)$, and the normaliser, centraliser, closure $\mathrm{cl}(H)$, derived and Frattini subgroups, $\mathrm{Fix}(H)$, and the pointwise and setwise stabilisers of $\mathrm{Fix}(H)$ each cost at most $C(H) + O(1)$.*

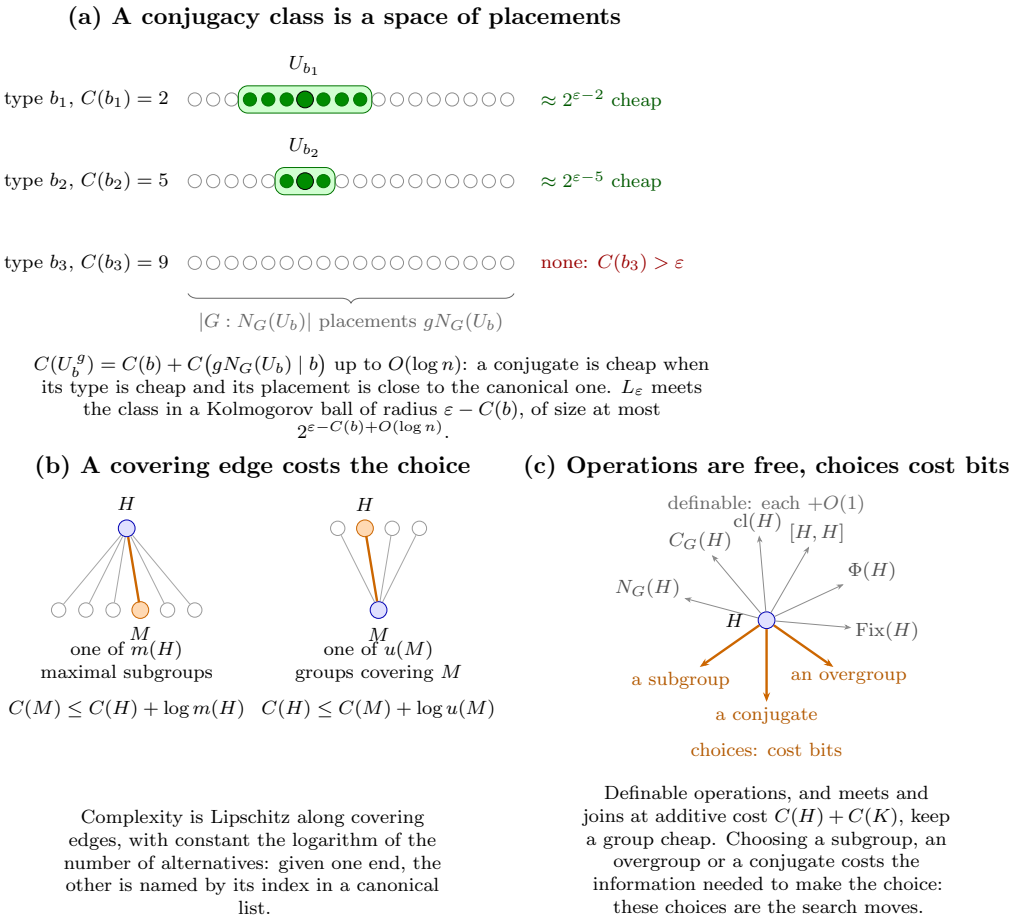


Figure 3: How cheap subgroups sit in the lattice. *(a)* A conjugacy class is the space of placements $G/N_G(U_b)$ of one type b. Its cheap members form a Kolmogorov ball around the canonical realisation U_b, of radius $\varepsilon - C(b)$: large for a cheap type, small for a costly one, and empty when $C(b) > \varepsilon$ (Theorem 6.4). *(b)* Along a covering edge $M < H$ the complexity changes by at most the logarithm of the number of alternatives, $m(H)$ maximal subgroups going down or $u(M)$ covering groups going up (Theorem 6.5). *(c)* Definable operations add only $O(1)$ (Theorem 6.6); the choice of a subgroup, an overgroup or a conjugate costs bits, and these choices are the moves of the search. The numbers in (a) are illustrative.

Proof. Each is computable from its arguments. □

L_ε is approximately closed under definable operations, with additive cost, and not closed under the *choice* of a subgroup, overgroup or conjugate. The choices are the search moves, and their cost is the information needed to make them.

6.4 Chain entropy

Definition 6.7 (Chain entropy and the strong sublattice)**.** Fix, computably from n, a finite set $\mathcal{S}$ of *seed* subgroups with a prefix-free code of lengths $c(R)$ (the root G is a seed with $c = O(1)$), and a finite alphabet $\Gamma \subseteq G$ of conjugating letters with a prefix-free code of lengths $\kappa(\gamma)$. The *move graph* has an edge $H \to M$ for each maximal subgroup M of H, of weight $\omega = 1 + \log r(H) + \log |H : N_H(M)|$, and an edge $H \to H^\gamma$ of weight $1 + \kappa(\gamma)$ for each $\gamma \in \Gamma$. The *chain entropy* $d_{\mathcal{S}}(H)$ is the least value of $c(R) + \sum \omega$ over seeds R and paths from R to H, and the *strong sublattice* of radius ε is $B_\varepsilon = \{H : d_{\mathcal{S}}(H) \le \varepsilon\}$.

The weights are code lengths of a two-stage code (class uniformly, then conjugate uniformly) with a flag bit distinguishing down moves from conjugations; at each node they define a sub-probability over outgoing edges.

Theorem 6.8 (Soundness of the strong sublattice)**.** *$C(H \mid n) \le d_{\mathcal{S}}(H) + O(\log n)$ for every H. Consequently: (i) $B_\varepsilon \subseteq L_{\varepsilon + O(\log n)}$ and $|B_\varepsilon| \le (\varepsilon+1)2^{\varepsilon}$; (ii) B_ε is decidable; (iii) every cell of every $H \in B_\varepsilon$ is an $(\varepsilon + O(\log n))$-strong model, and its profile points bound $h^G_{x,\varepsilon+O(\log n)}$ from above.*

Proof. Encode H by the seed code and the path, each step arithmetic-coded with the sub-probabilities at the current node, which the decoder can compute; the length is the total weight plus $O(1)$, plus $O(\log n)$ to delimit the path. For (i), the products of sub-probabilities along paths of a fixed length t sum to at most 1, so at most 2^{ε} such paths have weight at most ε, and $t \le \varepsilon$ since every edge weighs at least 1. (ii) follows, and (iii) is Theorem 6.2. □

Corollary 6.9 (The budgeted search lives in the strong sublattice)**.** *With the codes of Theorem 6.7, the states reachable from a group H with remaining budget b are the groups at chain-entropy distance at most b from H. The decision process of Section 7 is posed on B_ε, and every model it produces is $(\varepsilon + O(\log n))$-strong.*

The root alone is not enough. The first step down from the root costs at least the logarithm of the smallest index of a proper subgroup. For $\mathrm{Sym}(X)$ with $N \ge 5$ the only subgroup of index less than N is $\mathrm{Alt}(X)$, and $\mathrm{Alt}(X)$ has no proper subgroup of index less than N; for $\mathrm{GL}(n,2)$ with $n \ge 5$ the smallest index is $2^n - 1$ [12, Table 5.2.A]. In both cases every hypothesis other than the root (and $\mathrm{Alt}(X)$) has chain entropy at least about n bits from the root: nothing that could compress a string of length n is reachable. Chain entropy from the root treats every subset, block system, subspace and form as equally likely, and almost all of them are complex.

Completeness is relative to seeds and edge codes. B_ε misses cheap groups that are expensive to reach, and three remedies are compatible with Theorem 6.8: structured *seeds* with their own short codes, structured *edges* with named arguments, and a *learned prior* $-\log q(M \mid H)$, which also defines sub-probabilities. No choice gives $B_\varepsilon = L_\varepsilon$ (Theorem 6.3(i)), and by the maximal gap theorem every decidable family of $2^{\mathrm{poly}(n)}$ cells is blind to some strings. The choice of seeds and edge codes decides which symmetric structure counts as simple; Sections 10.4 and 11.3 make it for the two ambient groups.

7 The lattice as a decision process

7.1 Reinforcement-learning background

We recall the notions used below, following [31] and, for undiscounted problems, [4].

Markov decision processes. An MDP has states $\mathcal{S}$, actions $\mathcal{A}(s)$, transition probabilities $P(s' \mid s, a)$, rewards $r(s,a,s')$ and a discount $\gamma \in [0,1]$. An episode $s_0, a_0, r_1, s_1, \ldots, s_T$ ends at a terminal state. A policy $\pi(a \mid s)$ has *return* $G_t = \sum_{k \ge 0} \gamma^k r_{t+k+1}$, state value $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ and action value $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$. The optimal values satisfy the Bellman optimality equations

$$V^*(s) = \max_{a \in \mathcal{A}(s)} Q^*(s,a), \qquad Q^*(s,a) = \sum_{s'} P(s' \mid s,a)\big[r(s,a,s') + \gamma V^*(s')\big],$$

and a policy greedy with respect to Q^* is optimal.

Undiscounted episodic problems. If every policy reaches a terminal state (*proper* policies in stochastic shortest-path problems [4]), $\gamma = 1$ is well posed and V^* is the unique solution of the Bellman equations. Discounting would weight an improvement found k steps deeper by γ^k, which has no meaning for code lengths; we take $\gamma = 1$.

Deterministic transitions, afterstates and stopping. If $s' = f(s, a)$ then $Q^*(s, a) = r(s, a, f(s, a)) + V^*(f(s, a))$: it suffices to learn the value of the state reached after the action, the *afterstate* [31, §6.8], and to evaluate candidates by reward plus afterstate value. An optimal-stopping problem has an action STOP leading to a terminal state.

Potential-based rewards. If $r(s, a, s') = \Phi(s) - \Phi(s')$, the terminal reward is 0 and $\gamma = 1$, returns telescope, $G_t = \Phi(s_t) - \Phi(s_T)$: maximising the return is minimising the potential where the episode stops. This is the undiscounted case of potential-based shaping [19]; it leaves optimal policies unchanged and makes $V^\pi(s) = \Phi(s) - \mathbb{E}_\pi \Phi(s_T)$ the potential still to be removed.

Monte Carlo control and exploring starts. Monte Carlo prediction averages observed returns; Monte Carlo control alternates evaluation with greedy improvement. Convergence needs every state–action pair to keep being tried; *exploring starts* begin episodes at random states with a random first action [31, §5.3], and η-greedy policies act randomly with probability η.

Function approximation and replay. A parametric $\widehat{V}_\theta(\phi(s))$ is fitted to targets z_t by regression. Monte Carlo targets are unbiased; bootstrapped (temporal-difference) targets have lower variance but, with function approximation and off-policy data, can diverge [31, §11.3]. Replay buffers reuse and decorrelate data [16, 18].

Search. Monte Carlo tree search (MCTS) grows a search tree by repeated selection with a bandit rule, expansion, evaluation by rollouts or a learned value, and backup [13, 5]; AlphaZero couples it with a policy and value network trained on the search's own output [29]. We adapt MCTS to the lattice in Section 9.4.

7.2 States, actions and rewards

Definition 7.1 (Lattice decision process)**.** Fix a description scheme, a budget ε and a weight $\lambda \geq 0$.

- A *state* is $s = (w, x, \lambda, b)$: a word w with its group $H = H_w$, the string x, the weight, and the remaining budget $b = \varepsilon - |w|$.
- *Actions*:
  - DOWN(M) for a maximal subgroup M of H, with code Δw;
  - SIDE(γ) for a letter γ of a fixed alphabet $\Gamma \subseteq G$ of cheap elements (coordinate transpositions, bit flips, transvections), giving H^γ, with code Δw;
  - STOP.

  A move is feasible if $|\Delta w| \leq b$. An optional UP(π), the join $\langle H, \pi \rangle$ with a cheap letter, is discussed in Theorem 7.3.
- *Transitions* are deterministic: $w \mapsto w\Delta w$.

- *Rewards* are decreases of the potential of Theorem 5.5:

$$r(s, a, s') = \Phi_\lambda(s) - \Phi_\lambda(s') = \mathcal{L}_H(x) - \mathcal{L}_{H'}(x) - \lambda|\Delta w|,$$

  and STOP earns 0. The discount is $\gamma = 1$.

A positive reward means the move shortened the weighted description of x: the gain in $\mathcal{L}$ exceeds λ times the bits needed to describe the move. The return from s_t is $G_t = \Phi_\lambda(s_t) - \Phi_\lambda(s_T)$, the number of weighted bits removed between s_t and the stopping point. Figure 4 shows an episode, the moves available along it, and its trace in the description profile. For a state s_t, or a search node v, we abbreviate the codelength of x under its group as $\mathcal{L}_t$ or $\mathcal{L}_v$.

Lemma 7.2 (Termination)**.** *Every episode has at most ε moves, of which at most $\log |H_{w_0}|$ are down moves.*

Proof. Every move has a code of at least one bit; a down move at least halves the order. □

All policies are proper, so $\gamma = 1$ is well posed.

Starting states. The root G has cost $O(1)$ and every subgroup lies below it, so episodes may start at the root with a random first action; since the root alone reaches nothing compressive (Section 6), they may also start at any seed. Alternatively, episodes start at a *random subgroup*: in the present framework this means a random word w_0 of the scheme with $|w_0| \le \varepsilon$, whose cost is charged. A random conjugate of a large group by a uniformly random placement is not an admissible start, since the placement alone costs about n^2 bits in $\mathrm{GL}(n, 2)$ and up to about $n2^n$ bits in $\mathrm{Sym}(X)$.

With the codes of Theorem 6.7, the region a budgeted episode can explore is the chain-entropy ball around its start (Theorem 6.9): the decision process lives on the strong sublattice, and every model it produces is strong.

Remark 7.3 (Up moves)**.** With down and sideways moves only, the region explored from a start w_0 is the union of the down-sets of its cheap conjugates. Up moves $\langle H, \pi \rangle$ coarsen the partition (its orbits are joined with those of $\langle \pi \rangle$, since orb preserves joins) and can leave the isomorphism type of the start, but they create cycles in the move graph. We allow them only in the exploring starts phase, and keep the planner on the acyclic down-graph (Section 9.4).

8 Value functions on the lattice

This section is the core of the paper. We show that finding a model for x is the same as computing an optimal value on the lattice, describe that value precisely, and derive the bounds, dualities and symmetries that a learned value function should respect. Figure 5 summarises the two main results.

8.1 The optimal value and its Bellman recursion

For a state s let $\mathcal{R}(s)$ be the set of states reachable from s by feasible moves, s included, and $\mathcal{R}_\downarrow(s)$ the subset reachable by down moves only.

Proposition 8.1 (Value as best reachable model)**.** *Write $m(s) = \min_{s' \in \mathcal{R}(s)} \Phi_\lambda(s')$. Then*

$$V^*(s) = \Phi_\lambda(s) - m(s) \ \ge 0, \qquad Q^*(s, a) = \Phi_\lambda(s) - m(s_a),$$

and m satisfies the recursion

$$m(s) = \min \Big\{ \Phi_\lambda(s), \ \min_{a \ \textit{feasible}} m(s_a) \Big\},$$

which for down moves runs over the maximal subgroups of H_w. V^* depends on (H, x, λ, b) and not on the cost $|w|$ already spent.

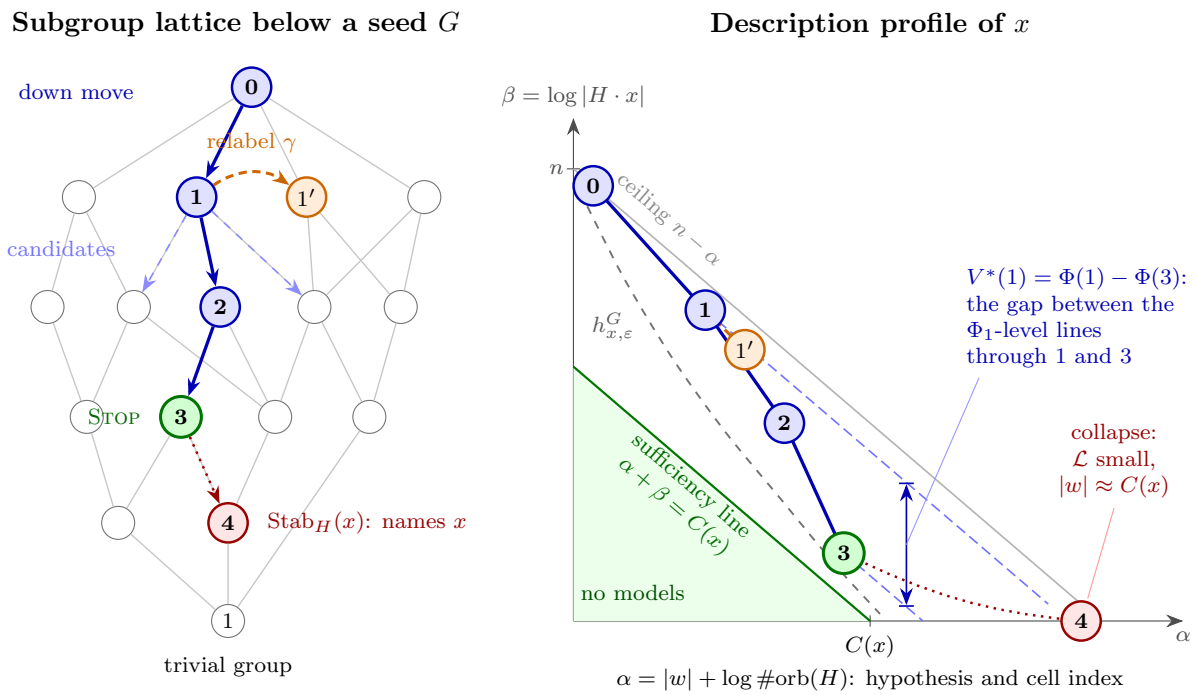


Figure 4: An episode of the lattice decision process and its image in the description profile. *Left:* part of the subgroup lattice below a seed G (node 0). From node 1 the agent has several feasible moves: DOWN to any maximal subgroup (solid and dashed blue arrows; the dashed ones are the candidates not taken), SIDE(γ) to an isomorphic conjugate $1' = H^\gamma$ (orange), or STOP. The agent goes $0 \to 1 \to 2 \to 3$ and stops at 3 (green). Continuing down would end at $\mathrm{Stab}_H(x)$ (node 4, red), where the codelength collapses to its minimum (Theorem 4.1) but the word has grown to about $C(x)$ bits: the stabiliser names x. *Right:* each state w is plotted at $(\alpha, \beta) = (|w| + \log \#\mathrm{orb}(H_w),\ \log|H_w \cdot x|)$. The true profile $h^G_{x,\varepsilon}$ (dashed) lies between the sufficiency line $\alpha + \beta = C(x)$ and the ceiling $n - \alpha$; no search can compute it, and every point found lies on or above it, a certified upper bound (up to $O(1)$ terms suppressed in the picture). Moves to the right pay for the description of the move, moves down shrink the orbit of x. Since $\Phi_1 = \alpha + \beta$, the level sets of the potential are the lines of slope -1, and the optimal value of a state is the vertical gap between its level line and the lowest level line reachable from it, here the one through 3.

Proof. The Bellman optimality equation with $\gamma = 1$, deterministic transitions and STOP worth 0 reads $V^*(s) = \max\{0, \max_a[\Phi(s) - \Phi(s_a) + V^*(s_a)]\}$. Substituting $V^* = \Phi - m$ gives the recursion; unrolling it gives the minimum over $\mathcal{R}(s)$. Every potential in $\mathcal{R}(s)$ contains $\lambda|w|$, which cancels in $\Phi(s) - m(s)$. □

So the value of a subgroup is the gap between its own model and the best model below it, and $Q^*(s, \mathrm{DOWN}(M))$ ranks the maximal subgroups by the best model in their down-sets. At the root,

$$\min_{s \in \mathcal{R}(\mathrm{root})} \Phi_\lambda(s) = \Phi_\lambda(\mathrm{root}) - V^*(\mathrm{root}) :$$

the best model of x in the hypothesis class is read off the value at the root. *Model search is value estimation.*

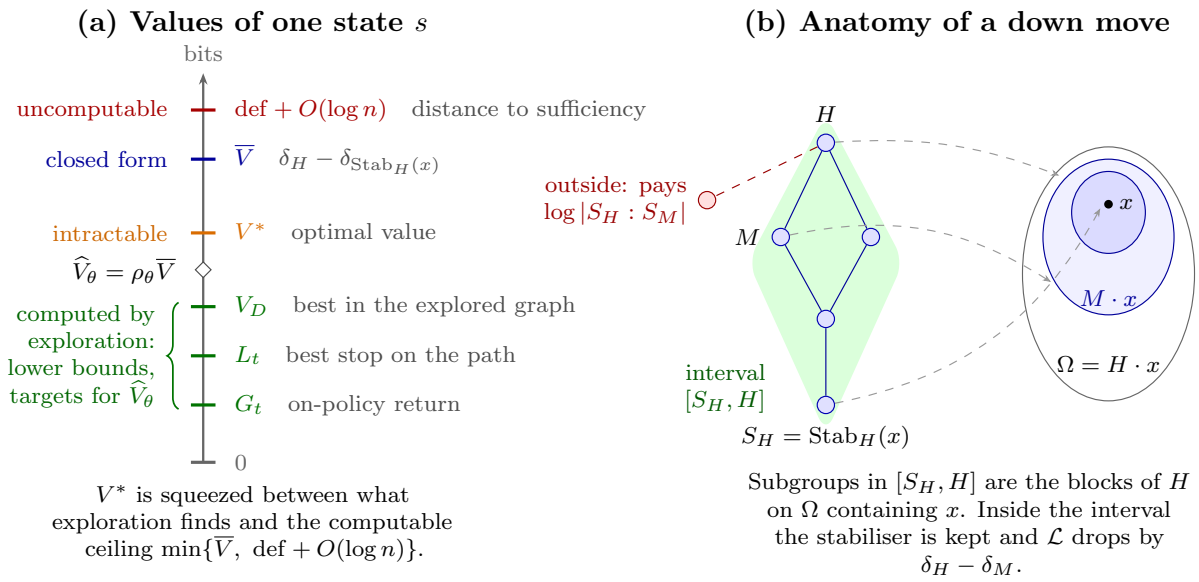


Figure 5: Value functions on the lattice. *(a)* For one state, the optimal value V^* lies between the targets found by exploration, $G_t \le L_t \le V_D$ (Theorem 9.1), and the ceiling formed by the closed-form bound $\overline{V} = \delta_H - \delta_{\mathrm{Stab}_H(x)}$ (Theorem 8.4) and the deficiency (Theorem 8.5); the colours mark the three levels of computability of Theorem 8.9, and the learned value $\widehat{V}_\theta = \rho_\theta \overline{V}$ respects the ceiling by construction. *(b)* The subgroups between $\mathrm{Stab}_H(x)$ and H correspond to the blocks of H on the orbit of x that contain x (Theorem 8.3); down moves inside this interval keep the stabiliser and lower $\mathcal{L}$ by the excess removed, while moves outside it also pay for the stabiliser they break (Theorem 8.2).

8.2 The anatomy of a down move

Proposition 8.2 (Reward decomposition)**.** *Let $M \le H$ and write $S_H = \mathrm{Stab}_H(x)$, $S_M = \mathrm{Stab}_M(x) = M \cap S_H$. Then*

$$\mathcal{L}_H(x) - \mathcal{L}_M(x) = \underbrace{(\delta_H - \delta_M)}_{\text{excess removed}} - \underbrace{\log |S_H : S_M|}_{\text{stabiliser lost}},$$

so the reward of Down(M) *is* $(\delta_H - \delta_M) - \log |S_H : S_M| - \lambda |\Delta w|$.

Proof. Theorem 3.4 for H and M. □

A down move gains by removing elements with large fixed spaces and loses by breaking the stabiliser of x. The moves that lose nothing form an interval.

Lemma 8.3 (The refinement interval)**.** *Let $\Omega = H \cdot x$. The maps $M \mapsto M \cdot x$ and $B \mapsto H_{\{B\}}$ (setwise stabiliser) are mutually inverse, inclusion-preserving bijections between the interval $[S_H, H]$ of the subgroup lattice and the set of blocks of imprimitivity of H on Ω that contain x. For $M \in [S_H, H]$,*

$$\mathrm{Stab}_M(x) = S_H, \qquad \mathcal{L}_M(x) = \mathcal{L}_H(x) - (\delta_H - \delta_M) \le \mathcal{L}_H(x),$$

and the cell of x shrinks from Ω to the block $M \cdot x$.

Proof. The correspondence between overgroups of a point stabiliser in a transitive action and blocks containing the point is standard [9, Thm. 1.5A]. For $M \ge S_H$, $\mathrm{Stab}_M(x) = M \cap S_H = S_H$, and Theorem 8.2 gives the formula. □

Moves inside $[S_H, H]$ are exactly the refinements of the cell of x that keep its stabiliser; they never increase $\mathcal{L}$, and the uncosted optimum S_H is the bottom of the interval (Theorem 4.1). A cheap symmetric refinement that halves the cell of x corresponds to a block of size $|\Omega|/2$ whose stabiliser has a short description: this is the halving question of [1, Question 7.2] in lattice form, and a search that stalls inside an interval with no cheap blocks may have reached a genuine plateau of h_x^G. Moves outside the interval pay $\log|S_H : S_M|$ and are worthwhile only if they remove enough excess.

8.3 Computable bounds

Theorem 8.4 (Value sandwich)**.** *For down moves only, for every $\lambda \geq 0$ and budget b,*

$$0 \;\leq\; V^*(H, x, \lambda, b) \;\leq\; \overline{V}(H, x) := \delta_H - \delta_{\mathrm{Stab}_H(x)},$$

and every state reachable from s has potential at least $\Phi_{\mathrm{low}}(s) = \lambda|w| + \log \#\mathrm{orb}(\mathrm{Stab}_H(x))$. Along down moves $\overline{V}$ is non-increasing and Φ_{low} is non-decreasing.

Proof. $V^* \geq 0$ because STOP is always available. Reachable states have groups $K \leq H$ and $|w'| \geq |w|$, so $m(s) \geq \lambda|w| + \min_{K \leq H} \mathcal{L}_K(x) = \Phi_{\mathrm{low}}(s)$ by Theorem 4.1, and $V^* \leq \mathcal{L}_H(x) - \log \#\mathrm{orb}(\mathrm{Stab}_H x) = \overline{V}$ by Theorem 4.2. Monotonicity: if $M \leq H$ then $\min_{K \leq M} \mathcal{L} \geq \min_{K \leq H} \mathcal{L}$, and $|w|$ only grows. □

Both bounds are computable from H and x: $\overline{V}$ needs $|H|$, $\#\mathrm{orb}(H)$, $\mathrm{Stab}_H(x)$ and $\#\mathrm{orb}(\mathrm{Stab}_H(x))$. They have three uses:

1. *Pruning.* A node with $\Phi_{\mathrm{low}}(s)$ at least the best potential found so far cannot lead to a better model (Section 9.4).
2. *A bounded value head.* The network predicts a fraction $\rho_\theta \in [0, 1]$ and outputs $\widehat{V} = \rho_\theta \overline{V}$, so that predictions always respect the sandwich.
3. *A uncosted baseline.* $\overline{V}$ is the exact value for $\lambda = 0$ and $b = \infty$; the learned part of the value is the effect of the cost.

With sideways moves the bound applies to each placement separately: by equivariance, $\mathcal{L}_{H^g}(x) = \mathcal{L}_H(g^{-1}x)$, so the reachable minimum is at least $\min_g \log \#\mathrm{orb}(\mathrm{Stab}_H(g^{-1}x))$ over the reachable placements g.

Proposition 8.5 (Cost removes the spurious value)**.** *Let $\lambda = 1$ and let $\mathrm{def}(s) = \Phi_1(s) - C(x)$ be the deficiency of the model at s. Then $V^*(s) \leq \mathrm{def}(s) + O(\log n)$. At the root G, with down moves and no budget,*

$$V^*_{\lambda=0}(G) = \mathcal{L}_G(x) - \log \#\mathrm{orb}(G_x) = \begin{cases} n - 1 & G = \mathrm{Sym}(X) \text{ or } \mathrm{AGL}(n, 2), \\ \log \frac{2(N-1)}{3} \approx n - 0.58 & G = \mathrm{GL}(n, 2),\ x \neq 0, \end{cases}$$

for every x, whereas $V^*_{\lambda=1}(G) \leq n - C(x) + O(\log n)$.

Proof. By Theorem 5.3, every reachable state has $\Phi_1 \geq C(x) - O(\log n)$, so $m(s) \geq C(x) - O(\log n)$. The uncosted value at the root is $\mathcal{L}_G(x) - \min_K \mathcal{L}_K(x)$ by Theorem 4.2, with the minimum from Theorem 4.3: $\mathcal{L}_{\mathrm{Sym}(X)}(x) = 0 + n$, $\mathcal{L}_{\mathrm{AGL}(n,2)}(x) = n$, and $\mathcal{L}_{\mathrm{GL}(n,2)}(x) = 1 + \log(N - 1)$ for $x \neq 0$. For the costed bound, $\Phi_1(G) = O(1) + \mathcal{L}_G(x) \leq n + O(1)$. □

Without a cost, every string has a root value of about n bits, all of it spurious. With the cost, incompressible strings have root value $O(\log n)$, and a large value certifies, up to $O(\log n)$, that x has short symmetric descriptions.

Proposition 8.6 (Planted strings)**.** *Let $P = H_{w_P}$ be reachable, let y be uniform on* $\mathrm{Fix}(P)$ *(minus* 0 *for* $\mathrm{GL}(n,2)$*), and $c \geq 0$. With probability at least $1 - 2^{-c}$,*

$$\mathrm{def}(w_P) \;\leq\; |w_P| + \big(\log \#\mathrm{orb}(P) - \log|\mathrm{Fix}(P)|\big) + c + O(\log n).$$

Proof. y is fixed, so $\mathcal{L}_P(y) = \log \#\mathrm{orb}(P)$ and $\Phi_1(w_P) = |w_P| + \log \#\mathrm{orb}(P)$. At most $2^{\log|\mathrm{Fix}(P)|-c}$ points of $\mathrm{Fix}(P)$ have $C(y \mid P) < \log|\mathrm{Fix}(P)| - c$, and $C(y) \geq C(y \mid P) - O(1)$. □

Since every point of $\mathrm{Fix}(P)$ is its own orbit, $\log \#\mathrm{orb}(P) \geq \log|\mathrm{Fix}(P)|$; the gap is small when P has few non-trivial orbits relative to its fixed points, as for H_2 in Theorem 11.2, where it is one bit. Planted strings therefore come with a known near-sufficient model and a known lower bound on the root value, which is what makes them usable for training and validation.

Remark 8.7 (Varying the weight)**.** For a state s let $h_x^{\downarrow s}(a)$ be the least codelength $\mathcal{L}_{H_{w'}}(x)$ over states $s' \in \mathcal{R}(s)$ with $|w'| - |w| \leq a$: the structure function of x restricted to hypotheses reachable from s with at most a further bits. Grouping the reachable states by a in Theorem 8.1 gives

$$V_\lambda^*(s) = \mathcal{L}_{H_w}(x) - \min_{a \geq 0} \big[\lambda a + h_x^{\downarrow s}(a)\big],$$

a Legendre transform. So the values for all λ together determine exactly the lower convex hull of $h_x^{\downarrow s}$: a search that sweeps λ recovers the convex part of the reachable profile and nothing more, and $\lambda = 1$ selects the shortest two-part code. A value network meant for many weights can use a *hull head*, predicting points (a_k, j_k) and setting $\widehat{V}_\lambda = \mathcal{L}_{H_w}(x) - \min_k(\lambda a_k + j_k)$, which has the right shape in λ by construction.

8.4 Symmetries of the value function

Proposition 8.8 (Equivariance)**.** *For $g \in G$, $\mathcal{L}_{H^g}(gx) = \mathcal{L}_H(x)$, $\overline{V}(H^g, gx) = \overline{V}(H, x)$ and Φ_{low} is invariant up to the cost term. If the description scheme is invariant under a subgroup $\Gamma \leq G$, in the sense that $|w^g| = |w|$ and $H_{w^g} = H_w^g$ for a relabelling $w \mapsto w^g$ of words, then $V^*(H^g, gx, \lambda, b) = V^*(H, x, \lambda, b)$ for $g \in \Gamma$.*

Proof. Conjugation by g is a lattice automorphism mapping orbits to orbits, stabilisers to stabilisers and ε_H to ε_{H^g}; for $g \in \Gamma$ it also preserves costs, so it maps $\mathcal{R}(s)$ onto $\mathcal{R}(gs)$ preserving potentials. □

The cost-free part of the value is a function of the G-orbit of the pair (H, x), i.e. of the conjugacy class of H together with the position of x relative to H. Invariant features available for every ambient group are: $\log|H|$, δ_H, $\log|\mathrm{Stab}_H(x)|$, $\delta_{\mathrm{Stab}_H(x)}$, $\overline{V}$, $\log \#\mathrm{orb}(H)$, $\log|\mathrm{Fix}(H)|$, $\log|\mathrm{Fix}(\mathrm{Stab}_H(x))|$, $[x \in \mathrm{Fix}(H)]$, the multiset of orbit sizes, the number of conjugacy classes of maximal subgroups and their indices, and the block structure of H on $H \cdot x$. Ambient-specific features are given in Sections 10.5 and 11.4: marks and double-coset counts in Burnside coordinates, module-theoretic invariants in $\mathrm{GL}(n,2)$. The cost part breaks the symmetry to Γ and is represented by the word w, read by a sequence model, with λ and b. Invariant features, canonical forms under Γ, Γ-equivariant networks, and raw encoders trained with augmentation are the options; the networks of Section 9.5 combine the last with the first.

8.5 Three levels of computability

Remark 8.9. The quantities above have different status.

(a) The *deficiency* $\mathrm{def}(s) = \Phi_1(s) - C(x)$, the distance to the sufficiency line, is uncomputable, because $C(x)$ is. No procedure certifies that a model is sufficient; searches give upper bounds on the profile and never matching lower bounds [1, Section 9.3].

(b) The *optimal value* V^* is computable in principle, as a minimum over the finite set $\mathcal{R}(s)$, but intractable. By Theorem 8.5 it is the part of the deficiency that the hypothesis class can remove; the remainder, the blindness of the class to the structure of x, is uncomputable as well, and by the maximal gap theorem it can be almost all of the deficiency.

(c) The *learned value* $\widehat{V}_\theta$ is cheap. Its targets are improvements realised on explored paths (Theorem 9.1), so

$$G_t \;\le\; L_t \;\le\; V^*(s_t) \;\le\; \min\big\{\overline{V}(s_t),\ \mathrm{def}(s_t) + O(\log n)\big\}.$$

with the second term of the minimum for $\lambda = 1$.

The value function estimates the distance to the sufficiency line in the only sense available: as the distance that can be travelled, estimated from below by exploring paths and from above by the computable $\overline{V}$. Planted strings are the only place where a near-sufficient reachable model is known to exist (Theorem 8.6); the compressor estimate $\hat{C}(x) \ge C(x)$ gives the only computable proxy $\Phi_1 - \hat{C}(x)$ for the deficiency, an underestimate, reported but never used as a reward.

8.6 Amortisation and training data

For a fixed x, the function $H \mapsto V^*(H, x, \lambda, b)$ is a property of one string. A value network trained across a distribution of strings is an *amortised* estimator: given (H, x, λ, b) it predicts the value without searching, and what it learns is where good models tend to lie for strings of that distribution. The training distribution should therefore be the one the search will meet. Because V^* is nearly zero for incompressible strings at $\lambda = 1$ (Theorem 8.5), it must also contain strings with structure the hypothesis class can see. Four sources play different roles.

- *Natural data* are the main training set. The certified targets of Theorem 9.1 need no ground truth, so training on natural strings is sound without knowing their best models. The learned prior of the training loop (Phase C of Algorithm 6) then becomes a prior over the symmetries that actually occur in such data. Symmetry reaches $\{0,1\}^n$ only through the encoding, so natural data should be encoded so that their symmetries act as permutations or linear maps of the coordinates: binary image patches, on which the lattice symmetries act by coordinate permutations; states of cellular automata and spin configurations; truth tables of Boolean functions, on which permutations of the variables act; and codewords of linear codes. Raw byte encodings of text or audio can be highly compressible and still show no symmetry in these coordinates, which is blindness caused by the encoding rather than absence of structure.

- *Planted strings* (Theorem 8.6) are the only strings with a near-sufficient reachable model known to exist. They serve for validation and calibration rather than as the main training set, since a network trained only on them learns the structure that was planted. They can be planted from groups that occur in the natural data, such as lattice translations and rotations or code automorphisms, rather than from random cheap groups.

- *Uniform strings* are a control, on which the network should predict values near zero.

- *Small lattices*, where Algorithm 8 computes V^* exactly, are the ground truth for value accuracy.

On natural data many searches will find little: by the maximal gap theorem of [1], blindness to symmetry is plausibly common. This is a finding about the data, not a failure of training, but it makes the learning signal sparse, and the planted and exact cases are what show that the search works when structure is there.

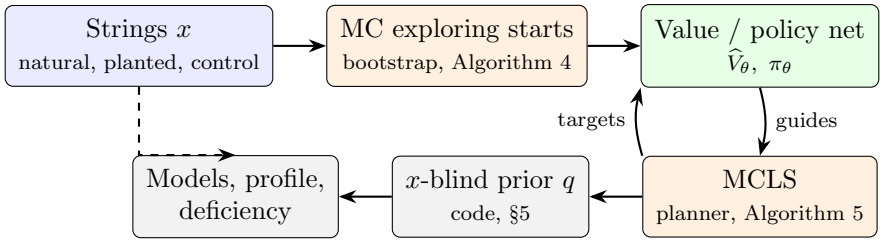


Figure 6: The pipeline. Monte Carlo with exploring starts trains a first value network; MCLS uses it to plan and returns improved targets; the descriptions found can be distilled into an x-blind prior that is then frozen and used as a code.

Algorithm 1 EVALUATE(w, x, λ)

Require: word w of the description scheme; string $x \neq 0$; weight λ

$H \leftarrow H_w$; $\quad |H| \leftarrow$ order of H

$\#\mathrm{orb}(H) \leftarrow \frac{1}{|H|} \sum_{C \in \mathrm{ccl}(H)} |C|\,|\mathrm{Fix}(c_C)|$ ▷ Burnside over classes

$\delta_H \leftarrow \log(\#\mathrm{orb}(H)|H|/N)$

$S \leftarrow \mathrm{Stab}_H(x)$; $\quad \#\mathrm{orb}(S)$ and δ_S likewise

$\mathcal{L} \leftarrow n + \delta_H - \log|S|$; $\quad \Phi \leftarrow \lambda|w| + \mathcal{L}$

$\overline{V} \leftarrow \delta_H - \delta_S$; $\quad \Phi_{\mathrm{low}} \leftarrow \lambda|w| + \log \#\mathrm{orb}(S)$ ▷ Theorem 8.4

$\phi \leftarrow$ invariant features of (H, x) (Section 8.4), with λ, b and w

return $\mathcal{L}, \Phi, \overline{V}, \Phi_{\mathrm{low}}, \phi$

9 Algorithms

Figure 6 shows how the pieces fit. A value network is bootstrapped by Monte Carlo with exploring starts; Monte Carlo lattice search uses it to plan, and its output in turn trains the network; an x-blind prior can be fitted to the descriptions found and frozen as a code.

9.1 Evaluating a state

No orbit on X is ever enumerated: $|H \cdot x| = |H|/|\mathrm{Stab}_H(x)|$ and #orb comes from conjugacy classes, so the cost is that of computing classes and a point stabiliser in H. In Burnside coordinates both reduce to tables of the abstract group (Section 10.5); in $\mathrm{GL}(n, 2)$, $|\mathrm{Fix}(c)| = 2^{n-\mathrm{rank}(c-I)}$ (Section 11.4).

9.2 Proposals

The number of maximal subgroups of a large H is enormous (the intransitive maximal subgroups $\mathrm{Sym}(B) \times \mathrm{Sym}(X \setminus B)$ of $\mathrm{Sym}(X)$ are as many as the subsets B; a parabolic subgroup of $\mathrm{GL}(n, 2)$ has about $2^{k(n-k)}$ conjugates), and random candidates almost never relate to x. Candidates are therefore generated from x, in four families, and each is charged its x-independent code length (Theorem 5.4).

The policy $\pi_\theta(a \mid s)$ sees x and ranks candidates; it is used for search and never for coding.

9.3 Monte Carlo with exploring starts

Certified targets. Training targets come from what was actually explored. Let D be a set of feasible words closed under prefixes, collected by any procedure and for any strings: the words of one episode, the union of many episodes, or the trees of MCLS. For $w \in D$ let D_w be the set of words of D that extend w, and for a string x and a weight λ put

$$V_D(w, x) = \Phi_\lambda(w, x) - \min_{u \in D_w} \Phi_\lambda(u, x),$$

Algorithm 2 PROPOSE(s, k)

Require: state $s = (w, x, \lambda, b)$ with $H = H_w$, $S = \mathrm{Stab}_H(x)$; number of candidates k

$\mathcal{C} \leftarrow \emptyset$

Interval moves: for maximal blocks $B \ni x$ of H on $H \cdot x$ add DOWN$(H_{\{B\}})$ ▷ Theorem 8.3; never increase $\mathcal{L}$

Stabiliser-preserving classes: for class representatives M_i of maximal subgroups, add conjugates M_i^u maximising $|M_i^u \cap S|$ over a few u

Structural moves: add ambient-specific candidates built from x (Sections 10.5 and 11.4), when expressible in the scheme

Sideways moves: add SIDE(γ) for letters $\gamma \in \Gamma$

Exploration: add random maximal subgroups from random classes

remove candidates with $|\Delta w| > b$; compute each code length $|\Delta w|$

return the best k candidates under the prior $\pi_\theta(\cdot \mid s)$, or all if no prior yet

Algorithm 3 SAMPLEPOINT$(\pi_{\mathrm{nat}}, \pi_0, R)$

Require: fractions π_{nat} of natural and π_0 of control strings; maximum planting depth R

with probability π_{nat}: **return** x drawn from an encoded natural corpus (Section 8.6)

with probability π_0: **return** x uniform on X ($X \setminus \{0\}$ for $\mathrm{GL}(n, 2)$) ▷ control

$w_P \leftarrow$ random word with at most R moves, from the root or a structured construction

$x \leftarrow$ uniform on $\mathrm{Fix}(H_{w_P})$, excluding 0 for $\mathrm{GL}(n, 2)$ (resample if empty)

with probability $\frac{1}{2}$ flip a few random coordinates of x ▷ non-singleton targets

return x, and w_P for diagnostics

the best improvement on w found in D.

Lemma 9.1 (Certified targets)**.** *For every D, every string x and every λ, with $s = (w, x, \lambda, \varepsilon - |w|)$,*

$$0 \;\le\; V_D(w, x) \;\le\; V^*(s).$$

V_D is monotone in D, and $V_D = \Phi_\lambda - m_D$ with $m_D(w, x) = \min\{\Phi_\lambda(w, x),\ \min_c m_D(c, x)\}$ over the children c of w in D.

Proof. Every $u \in D_w$ is reached from w by recorded moves, and $|u| \le \varepsilon$ because the words of D are feasible; the moves and their codes do not depend on the string or on the policy that recorded them. Hence $\Phi_\lambda(w, x) - \Phi_\lambda(u, x)$ is the return, for the string x, of a feasible policy that follows those moves and stops at u, and is at most $V^*(s)$ by Theorem 8.1. Taking the minimum over u gives $V_D \le V^*$, and $u = w$ gives $V_D \ge 0$. Monotonicity and the recursion are immediate from the definition. □

For an episode $s_0, \ldots, s_T$ with potentials Φ_t, take D to be the prefixes of its word. Then $V_D(w_t, x)$ is the gain to the best stopping point later on the path,

$$L_t = \Phi_t - \min_{t \le t' \le T} \Phi_{t'}, \qquad L_t = \max\big(0,\, r_{t+1} + L_{t+1}\big),\ L_T = 0,$$

computed in one backward pass. By the lemma, L_t is a lower bound on $V^*(s_t)$ for data from *any* policy, and it stays valid when the data are replayed under later policies. For down moves the value sandwich (Theorem 8.4) adds the ceiling $V^*(s_t) \le \overline{V}(s_t)$; with sideways moves, $\overline{V}$ is taken over the reachable placements. The on-policy return $G_t = \Phi_t - \Phi_T$ satisfies $G_t \le L_t$ but is not used as a target: it credits s_t with where this episode happened to stop, and it can be negative, which no value can be. We regress $\widehat{V}_\theta = \rho_\theta \overline{V}$ on L_t with a Huber loss. This phase needs no search and is used until the network is good enough to guide MCLS.

Algorithm 4 EPISODE($\widehat{V}_\theta, \eta$)

Require: value network; exploration rate η

$x \leftarrow$ SAMPLEPOINT; $\lambda \leftarrow$ uniform on a grid; $b \leftarrow \varepsilon$

$w \leftarrow$ root, or a random admissible start word w_0 (charged) ▷ exploring start

$t \leftarrow 0$; $\mathcal{T} \leftarrow [\,]$

loop

  $(\mathcal{L}_t, \Phi_t, \overline{V}_t, \cdot, \phi_t) \leftarrow$ EVALUATE(w, x, λ)

  $\mathcal{C} \leftarrow$ PROPOSE$(s_t, k) \cup \{$STOP$\}$

  if $t = 0$ **then** $a_t \leftarrow$ uniform on $\mathcal{C}$ ▷ exploring first action

  else if with probability η **then** $a_t \leftarrow$ uniform on $\mathcal{C}$

  else $a_t \leftarrow \arg\max_{a \in \mathcal{C}} \left[\Phi_t - \Phi(s_a) + \widehat{V}_\theta(s_a)\right]$, with STOP scoring 0

  append $(\phi_t, \Phi_t, \overline{V}_t)$ to $\mathcal{T}$

  if $a_t =$ STOP **then break**

  $w \leftarrow w\,\Delta w(a_t)$; $t \leftarrow t + 1$

return $\mathcal{T}$

The first round: random-policy data. Before any network exists, Phase A is run with a uniformly random policy: episodes are random down-chains from random starting words, and the data are the states visited with their potentials. A value network is trained on these data and then used, with exploration, to generate the next round of episodes; training and data collection then alternate. This is generalised policy iteration [31, §4.6], with Monte Carlo evaluation followed by greedy improvement. Two details decide whether the first round is informative.

1. *The target.* If the random policy has no STOP action, every chain runs to the trivial group, where $\mathcal{L} = n$, and with $\gamma = 1$ the return telescopes to $G_t = \Phi_t - \lambda|w_T| - n$. Apart from the cost of the random path, this is $\mathcal{L}_{H_{w_t}}(x) - n$, which is already computable exactly and says nothing about the subgroups below H_{w_t}. The best-stopping-point target L_t does not have this defect: by Theorem 9.1 it is a lower bound on $V^*(s_t)$ for data from *any* policy, including the random one, so the random episodes already give valid targets for the optimal value. In later rounds, better policies find better stopping points and so give larger L_t, while every round's L_t remains a lower bound on V^*.

2. *What to record.* Episodes should store the potential Φ_t (equivalently $\mathcal{L}_t$ and $|w_t|$) at every visited state, not only a return computed from them. Targets of any kind, G_t, L_t or others, can then be recomputed from stored data when the reward, the weight λ or the cost scheme changes, without regenerating episodes.

With an uncosted reward ($\lambda = 0$, $b = \infty$) the loop has nothing to learn, because the optimal value is $\delta_H - \delta_{\mathrm{Stab}_H(x)}$ in closed form (Theorem 4.2) and an improving policy simply heads for the stabiliser of x. The iteration is informative only once the cost term or a budget is part of the reward.

9.4 Monte Carlo lattice search

Monte Carlo lattice search (MCLS) is Monte Carlo tree search adapted to the subgroup lattice. Four features distinguish it from standard MCTS (Table 1). Figure 7 shows one simulation.

Path-independent backups. With transpositions a subgroup may be reached by paths of different costs, so a node should not store a potential, which depends on the path. It stores the *local optimum* $\mu_v = \min_{v' \text{ below } v}[\lambda\,\mathrm{cost}(v \to v') + \mathcal{L}_{v'}]$, the quantity $m - \lambda|w|$ of Theorem 8.1,

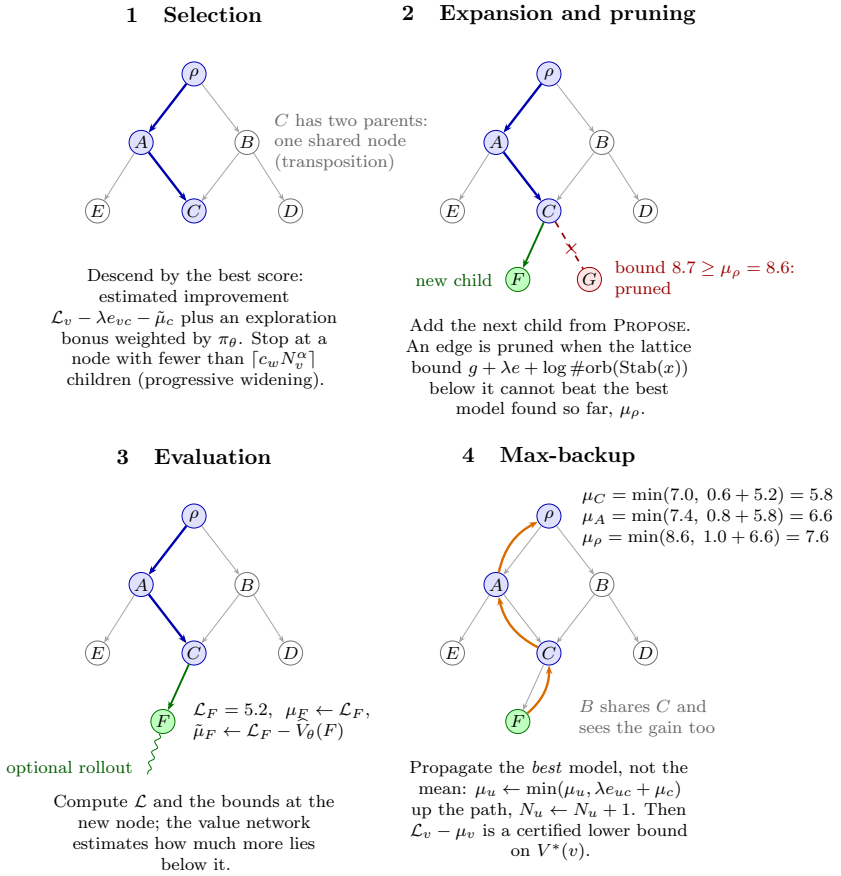


Figure 7: One simulation of Monte Carlo lattice search (Algorithm 5) on a small part of the lattice; the subgroup C is reached from both A and B and stored once. *1:* the search descends through stored nodes by the selection score, until it reaches a node that progressive widening allows to grow. *2:* a proposed child F is added; a candidate G whose lattice bound cannot beat the best potential found so far, μ_ρ, is pruned. *3:* the new node is evaluated: its codelength is exact, and the value network estimates the improvement still available below it. *4:* the best potential found below each node is passed up the path by minima, not averages; the shared node C passes the gain to every parent. The realised values $\mathcal{L}_v - \mu_v$ are certified lower bounds on $V^*(v)$ (Theorem 9.2). The numbers are illustrative, with $\lambda = 1$.

which depends only on the node. Then $V^*(v) = \mathcal{L}_v - \mu_v^*$, an edge $v \to c$ of cost e_{vc} has $Q^*(v, c) = \mathcal{L}_v - \lambda e_{vc} - \mu_c^*$, and the recursion $\mu_v = \min\{\mathcal{L}_v, \min_c(\lambda e_{vc} + \mu_c)\}$ is valid on any DAG. MCLS keeps two versions: a *realised* μ_v, computed from explored nodes only, so that $\mathcal{L}_v - \mu_v$ is a certified lower bound on $V^*(v)$; and an *estimated* $\tilde{\mu}_v$, in which unexplored leaves contribute $\mathcal{L}_\ell - \widehat{V}_\theta(\ell)$.

Proposition 9.2 (Soundness of MCLS)**.** *At every stage of Algorithm 5, for every node v, $\mu_v \geq \mu_v^*$, so $\mathcal{L}_v - \mu_v \leq V^*(v)$; the value μ_ρ is the potential of an actual hypothesis, hence an upper bound on the best model at the root; and a pruned edge cannot lead to a hypothesis with potential below μ_ρ.*

Proof. μ_v is initialised to $\mathcal{L}_v$ and only lowered to values $\lambda e + \mu_c$ realised by explored edges, so by induction it is the local value of an actual descendant. Any hypothesis reached through the pruned edge along the current path has potential at least $g + \lambda e_{vc}$ plus the subtree bound of Theorem 8.4 for c, which is at least μ_ρ. The subtree remains reachable through other edges, which are tested separately. □

	Standard MCTS	MCLS
Search space	tree of move sequences	DAG of subgroups (down moves); one node per subgroup, shared across paths (transpositions [7, 26])
Objective	expected terminal return	best model anywhere below the root (single-agent, deterministic)
Backup	mean of returns	minimum of local profile values, path-independent: $\mu_v = \min\{\mathcal{L}_v, \min_c(\lambda e_{vc} + \mu_c)\}$ [27]
Leaf evaluation	random rollout or value net	value net $\widehat{V}_\theta$, optionally a random down-chain rollout
Branching	enumerated moves	progressive widening from x-guided proposals [8]
Pruning	none	lattice bound Φ_{low} of Theorem 8.4
Output	best first move	archive of all models visited and the best one

Table 1: Standard MCTS versus Monte Carlo lattice search.

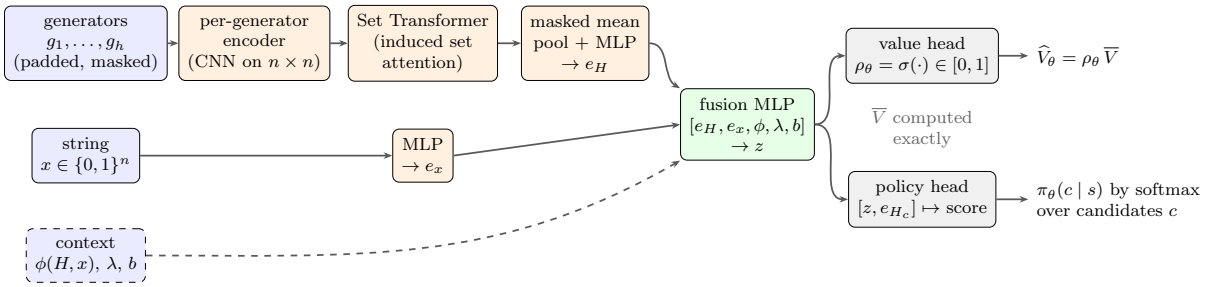


Figure 8: The value and policy networks. A generating set of H is encoded generator by generator and mixed by a Set Transformer, so the result does not depend on the order of the generators; the string is encoded separately; an optional context of invariant features and the weight and budget joins them in a fusion network. The value head predicts the fraction ρ_θ of the computable ceiling $\overline{V}$, so that every prediction respects the value sandwich; the policy head scores each candidate child.

Max-backups are appropriate because the problem is single-agent and deterministic [27]: there is no opponent and no chance, and the quantity of interest is the best model found, not an average. The exploration bonus keeps revisiting branches whose estimates are uncertain, so an early underestimate by $\widehat{V}_\theta$ is corrected by what the search actually finds. With a hard budget, nodes are keyed by subgroup and the cheapest cost found; a cheaper path to an existing node dominates and replaces the old one. MCLS also supplies training targets: $\mathcal{L}_v - \mu_v$ for every expanded node (a certified lower bound on V^*) and the visit distribution of its children for the policy π_θ, as in AlphaZero [29].

9.5 The value and policy networks

The networks map a state s to a value $\widehat{V}_\theta(s)$ and, for the planner, to scores $\pi_\theta(c \mid s)$ for its candidate moves. Three choices define them: how the hypothesis is presented, how the string is presented, and how the structure of Section 8 is respected. Figure 8 shows the architecture.

Inputs.

Algorithm 5 MCLS$(x, \lambda, \varepsilon, K_{\mathrm{sim}})$

Require: string x; weight λ; budget ε; number of simulations K_{sim}

table $\mathcal{T} \leftarrow \emptyset$ keyed by a canonical key of the (closed) subgroup
create root ρ: $\mathcal{L}_\rho$, $\mu_\rho \leftarrow \mathcal{L}_\rho$, $\tilde{\mu}_\rho \leftarrow \mathcal{L}_\rho - \widehat{V}_\theta(\rho)$, $N_\rho \leftarrow 0$
for $k = 1, \dots, K_{\mathrm{sim}}$ **do**
    $v \leftarrow \rho$; $g \leftarrow 0$; path $\leftarrow [\rho]$ ▷ g: cost spent from the root
    loop ▷ selection
        if $|\mathrm{children}(v)| < \lceil c_w N_v^{\alpha} \rceil$ **then break** ▷ progressive widening
        $c \leftarrow \arg\max_c \Big[\dfrac{\mathcal{L}_v - \lambda e_{vc} - \tilde{\mu}_c}{\max(\overline{V}_v, 1)} + c_{\mathrm{puct}}\, \pi_\theta(c \mid v) \dfrac{\sqrt{N_v}}{1 + N_c} \Big]$ over unpruned children
        $g \leftarrow g + \lambda e_{vc}$; $v \leftarrow c$; append v to path
        if v has no feasible move **then break**
    $a \leftarrow$ next untried candidate of PROPOSE$(v, \cdot)$ ▷ expansion
    $c \leftarrow$ lookup or create the node of H_v's child under a; record edge cost e_{vc}
    if $g + \lambda e_{vc} + \log \#\mathrm{orb}(\mathrm{Stab}_{H_c}(x)) \geq \mu_\rho$ **then** mark edge pruned; **continue** ▷ Theorem 8.4
    if c is new **then** evaluate c (Algorithm 1); $\mu_c \leftarrow \mathcal{L}_c$; $\tilde{\mu}_c \leftarrow \mathcal{L}_c - \widehat{V}_\theta(c)$
    optionally run a random down-chain from c and lower μ_c to the best value it realises
    for u on the path, from v back to ρ **do** ▷ max-backup of improvements
        $\mu_u \leftarrow \min\big(\mu_u,\ \min_{c'}(\lambda e_{uc'} + \mu_{c'})\big)$; $\tilde{\mu}_u \leftarrow \min\big(\tilde{\mu}_u,\ \min_{c'}(\lambda e_{uc'} + \tilde{\mu}_{c'})\big)$
        $N_u \leftarrow N_u + 1$
return the archive of all nodes with their profile points, and the node realising μ_ρ

- *The hypothesis.* H is presented by a generating set $\{g_1, \dots, g_h\}$, padded to a fixed maximum $h_{\max}$ and masked. In $\mathrm{GL}(n, 2)$ each generator is an $n \times n$ binary matrix. In Burnside coordinates the generating set is replaced by the type, an identifier of the abstract group from the library together with the multiplicities a_i, and the placement is read from the word w.
- *The string.* $x \in \{0, 1\}^n$, as a raw binary vector.
- *Context (optional).* The invariant features $\phi(H, x)$ of Section 8.4, which EVALUATE (Algorithm 1) computes anyway, and the weight λ and remaining budget b when one network serves several of them.

Architecture.

1. *Generator encoder.* Each generator is mapped to an embedding by a small convolutional network on its $n \times n$ matrix: convolution blocks, global average pooling and a linear projection.
2. *Set encoder.* A generating set has no order, so the generator embeddings are mixed by a Set Transformer [15], whose induced set attention blocks attend through a fixed number of learned inducing points. Masked mean pooling over the h generators and an MLP give a group embedding e_H that does not depend on the order of the generators.
3. *String encoder.* An MLP maps x to an embedding e_x.
4. *Fusion.* An MLP maps $[e_H, e_x, \phi, \lambda, b]$ to a trunk vector z.
5. *Heads.* The value head outputs $\rho_\theta = \sigma(\,\cdot\,) \in [0, 1]$, and $\widehat{V}_\theta = \rho_\theta\, \overline{V}$ (Theorem 8.4); for many weights at once the hull head of Theorem 8.7 can replace it. The policy head scores each candidate child c by an MLP on $[z, e_{H_c}]$, and $\pi_\theta(\cdot \mid s)$ is the softmax over the candidates.

A second design serialises the generators and the string as token streams, encodes each with its own Transformer and fuses them with further Transformer layers. It gives up the inductive bias of the set encoder for capacity.

What the encoders respect, and what they must learn.

- *Order of the generators.* Respected by construction.
- *Choice of generating set.* Two generating sets of the same group are different inputs. Training data are augmented by Nielsen moves ($g_i \mapsto g_i g_j$, $g_i \mapsto g_i^{-1}$) and by redundant generators, which preserve the group.
- *Equivariance.* Raw encoders do not enforce $V(H^g, gx) = V(H, x)$ for $g \in \Gamma$ (Theorem 8.8). Training data are augmented by random relabellings $g \in \Gamma$ applied to the generators and to x together, and the invariant features ϕ carry the invariant part exactly.
- *The value sandwich.* Enforced by the bounded head at no cost, since $\overline{V}$ is computed exactly when the state is evaluated. An unconstrained scalar output can violate it, and the bounded head also makes the target scale-free.

Targets and losses. The value head is trained with a Huber loss on certified targets: L_t from episodes (Theorem 9.1), $\mathcal{L}_v - \mu_v$ from MCLS (Theorem 9.2), and V_D for the strings that guided the exploration; for relabelled strings only predictions outside $[V_D, \overline{V}]$ are penalised. The regression is on the fraction, with target $V/\overline{V}$ when $\overline{V} > 0$. The on-policy return G_t is not used: for random-policy data it reflects where the random chain stopped, not what lies below the state (Section 9.3). The policy head is trained with a cross-entropy loss against the visit distribution of the children in MCLS.

Implementation. For $\mathrm{GL}(40, 2)$ the first design takes generating sets of up to 15 matrices. Each matrix passes through three pairs of 3×3 convolutions ($64, 128, 256$ channels) to a 256-dimensional embedding; four Set Transformer layers with 8 heads and 32 inducing points follow; the string MLP has widths $512, 512, 256$; the fusion MLP has widths $512, 256, 128$; dropout is 0.1. It is trained with AdamW (learning rate $3 \cdot 10^{-4}$, weight decay 10^{-5}), cosine annealing with warm restarts, mixed precision and early stopping on the validation loss. The token-stream design uses Transformers of width 768 with 12 heads, ten layers per stream and four fusion layers, about 10^8 parameters, trained with a Huber loss and AdamW at $5 \cdot 10^{-5}$ with per-module learning rates and a plateau schedule. Training triples $(x, \{g_i\}, \text{target})$ are produced by the episodes and searches of this section.

9.6 The training loop

Phase A needs no search and gives the first estimates. Phase B is generalised policy iteration with search as the improvement operator: MCLS with the current network finds better models than the network alone, and training on what it finds improves the network. Phase C changes the code itself: hypotheses that were useful across many strings become cheaper to describe, which shifts the trade-off of Theorem 8.7 in their favour. Because q is fitted on training strings and frozen, codes remain valid on test strings.

Algorithm 6 TRAIN

Phase A (bootstrap). Round 0: random-policy episodes; record Φ_t at every state
  repeat: regress $\widehat{V}_\theta = \rho_\theta \overline{V}$ on the targets L_t; run EPISODE (Algorithm 4) with the new $\widehat{V}_\theta$ on natural, planted and control strings
Phase B (search and distil). **repeat**
  for a batch of strings run MCLS with the current $\widehat{V}_\theta, \pi_\theta$
  value targets $\mathcal{L}_v - \mu_v$ for expanded nodes; policy targets from child visit counts
  insert the searched nodes into the shared graph and add the relabelled targets V_D (Theorem 9.1) for the whole batch
  update θ on a replay buffer of these targets
until validation regret stops improving (Section 9.8)
Phase C (optional, amortised MDL). Fit an x-blind prior q to the move sequences of the best descriptions found; freeze q; use c_q as the code and repeat Phase B

9.7 Sharing exploration across strings

A hypothesis is a coding scheme for every string at once, and this makes the expensive part of the search independent of x. Write the potential as

$$\Phi_\lambda(w, x) = \underbrace{\lambda|w| + \log \#\mathrm{orb}(H_w)}_{a_\lambda(w)} + \underbrace{\log |H_w \cdot x|}_{\beta(w,x)}.$$

The lattice, the moves, their codes, the cost $|w|$ and the term $\log \#\mathrm{orb}(H_w)$ (the α-coordinate of the node in the profile, up to $(1-\lambda)|w|$) are the same for all strings; only the height $\beta(w, x)$ depends on x. The group computations that dominate the cost of a search (maximal subgroups, normalisers, codes and the Burnside sum for $\#\mathrm{orb}(H_w)$) are therefore done once per node. The height costs one stabiliser computation per pair (w, x); it is the size of the cell containing x in Burnside coordinates, and a table lookup when the orbit labels of all points of X fit in memory.

Relabelled targets. Theorem 9.1 holds for every string, not only for the one that guided the exploration: an explored graph D gives the certified bound $V_D(\cdot, x)$ for any x, by one bottom-up pass with the potentials of x. This is the analogue of hindsight experience replay [2], with the string in the role of the goal and $\widehat{V}_\theta(w, x)$ a universal value function approximator [25]. The on-policy return G_t does not transfer, because its stopping point was chosen for the string that guided the episode. With transpositions, D is a DAG of groups and the same bound is computed by the path-independent recursion of Section 9.4, $V_D(v, x) = \mathcal{L}_v(x) - \mu_v^D(x)$ with $\mu_v^D(x) = \min\{\mathcal{L}_v(x), \min_c(\lambda e_{vc} + \mu_c^D(x))\}$ over explored children, for the Lagrangian problem; with a budget, the recursion is run on the trie of words.

Parallelism. Algorithm 7 separates four kinds of work. The actors expand the lattice; their work is independent of the strings, and each group computation is reused B times. The scorers compute heights for (node, string) pairs, an embarrassingly parallel task. The DP costs $O(|D| \cdot B \cdot |\Lambda|)$ arithmetic operations and vectorises over strings and weights. The learner sees B targets for every node explored instead of one. Changing λ or the budget needs only a new DP pass on stored data, as in Section 9.3. By Theorem 8.8, pairs (w, x) and (w^g, gx) with $g \in \Gamma$ carry the same values, so training pairs can be reduced to orbit representatives; and since $\mathcal{L}_H(x)$ is constant on H-orbits, strings in the same orbit of the node share their heights.

Limits. The bound $V_D(\cdot, x)$ is valid for every string but tight only for strings whose good models lie in D. Exploration guided by one string leaves the regions useful to another unexplored, and there the relabelled target is pessimistic rather than wrong; hence the regression on V_D

Algorithm 7 SHAREDTRAIN($\mathcal{X}, \Lambda$)

Require: a batch $\mathcal{X} = \{x_1, \dots, x_B\}$ of strings; a grid Λ of weights

$D \leftarrow$ seeds; for each new node w store $|w|$ and $\log \#\mathrm{orb}(H_w)$ ▷ independent of x

repeat

  actors, in parallel: pick a guide $x \in \mathcal{X}$ (or a batch criterion); run EPISODE or MCLS guided by $\widehat{V}_\theta(\cdot, x)$; insert the visited words into D

  scorers, in parallel: for every new node w and every $x \in \mathcal{X}$, compute $\beta(w, x) = \log |H_w \cdot x|$ and $\overline{V}(w, x)$

  DP: for every $x \in \mathcal{X}$ and $\lambda \in \Lambda$, one bottom-up pass over D gives $V_D(w, x)$ for all w (Theorem 9.1)

  learner: sample triples (w, x, λ); regress $\widehat{V}_\theta$ on V_D for the guide strings, and penalise it outside $[V_D, \overline{V}]$ for the others

until validation regret stops improving

Algorithm 8 EXACTVALUE(x, λ)

Require: all subgroups below a root, with their maximal subgroups and edge code lengths e_{HM}

for subgroups H in increasing order of $|H|$ **do**

  $\mathcal{L}_H \leftarrow \mathcal{L}_H(x)$ by Theorem 3.4

  $\mu^*_H \leftarrow \min\left\{\mathcal{L}_H,\ \min_{M \text{ maximal in } H}(\lambda e_{HM} + \mu^*_M)\right\}$

return $V^*(H) = \mathcal{L}_H - \mu^*_H$ for all H

only for guide strings and the one-sided penalty for the others. Exploration can be steered by a batch criterion, for instance an upper confidence bound on the largest or an upper quantile of the predicted values across the batch, or by clustering strings with similar structure. Sharing is strongest near the top of the lattice: coarse groups are useful for many strings, while deep nodes become specific to one string, and near $\mathrm{Stab}_H(x)$ every node in effect names x. The shared graph plays the role of an opening book; the end of each search remains a search for one string, by MCLS at test time.

9.8 Exact values on small lattices

When the whole lattice below a root is available, optimal values are computed exactly by dynamic programming over its Hasse diagram, from the bottom up. This is the ground truth against which learned values and search are validated: on $\mathrm{Sym}(\mathbb{F}_2^2) = S_4$, with 30 subgroups; in Burnside coordinates, below any hypothesis with a small abstract group, whose down-set is $\mathrm{Sub}(H)$ for every n (Theorem 10.6); and on $\mathrm{GL}(3, 2)$, with 179 subgroups, and $\mathrm{GL}(4, 2) \cong A_8$.

The validation protocol compares, over all pairs (H, x): value accuracy (error and rank correlation between $\widehat{V}_\theta$ and V^*, and the fraction of predictions outside the sandwich of Theorem 8.4, zero for the bounded head); search regret $\mu_\rho - \mu^*_{\mathrm{root}}$ as a function of simulations, for MCLS and for Monte Carlo episodes; profile recovery (the distance between the convex hull of the archive and the exact hull of Theorem 8.7); and, for $\lambda = 0$, the identity $V^* = \delta_H - \delta_{\mathrm{Stab}_H(x)}$ of Theorem 4.2, which tests the implementation of $\mathcal{L}$.

10 All permutations: Burnside coordinates

10.1 The ambient group $\mathrm{Sym}(X)$

With ambient group $\mathrm{Sym}(X)$ every partition of X is symmetric and every non-empty set is a cell. By [1, Theorem 5.1] this has two consequences: the symmetric structure function is

Kolmogorov's, $h_x^{\mathrm{Sym}} = h_x$, and its strong version is the strong structure function, $h_{x,\varepsilon}^{\mathrm{Sym}} = h_{x,\varepsilon}^{\mathrm{str}}$ up to $O(\varepsilon + \log n)$. The search in this section is therefore a search for strong models of every kind. The following makes the reduction explicit.

Proposition 10.1 (Two-cell realisation)**.** *For $x \in B \subsetneq X$ let $Y_B = \mathrm{Sym}(B) \times \mathrm{Sym}(X \setminus B)$. Then $\mathrm{orb}(Y_B) = \{B, X \setminus B\}$, $\mathcal{L}_{Y_B}(x) = 1 + \log|B|$, and $C(Y_B) = C(B) \pm O(1)$. Hence every finite-set model B of x is realised by a permutation hypothesis with total code length $C(Y_B) + \mathcal{L}_{Y_B}(x) = C(B) + \log|B| + O(1)$.*

Proof. Y_B preserves B and is transitive on it and on its complement. Y_B is computable from B; conversely B is one of the two orbits of Y_B, named by one bit. □

So with the cost included, the permutation search is complete: it can express every model, and by Theorem 5.3 it cannot do better than the best of them. Two caveats qualify this. Completeness is a property of the class, not of a search: reaching Y_B requires describing B cheaply, which a universal scheme can do but cannot search, while a structured scheme can be searched but names only some sets [1, Section 9.3]. And placement dominates: the type of Y_B costs $O(n)$ bits and its placement $C(B) \pm O(n)$ [1, Proposition 9.4], so a permutation hypothesis is informative about *symmetry* only when its information lies in a type with genuine group structure. Since the closure of a group can destroy that structure (Theorem 10.7), this section works with *descriptions* of groups in coordinates, never with closures.

10.2 The objective in Burnside coordinates

Setting. H is a small abstract group with representatives $K_1, \dots, K_r$ of its conjugacy classes of subgroups, $b = \sum_j a_j [H/K_j]$ a faithful type of degree N, ρ_b its canonical realisation on X and $U = \Phi_H(b, \sigma) = \sigma \rho_b(H) \sigma^{-1}$. Write $|b| = \sum_j a_j$ for the number of summands. For $x \in X$ let $y = \sigma^{-1} x$; it lies in a summand of type $j(x)$, at the coset $h_0 K_{j(x)}$, and we put $K_x = h_0 K_{j(x)} h_0^{-1}$, the stabiliser of that coset.

Proposition 10.2 ($\mathcal{L}$ in coordinates)**.**

$$\#\mathrm{orb}(U) = |b|, \qquad |U \cdot x| = |H : K_{j(x)}|, \qquad \mathrm{Stab}_U(x) = \sigma \rho_b(K_x) \sigma^{-1},$$

$$\mathcal{L}_U(x) = \log|b| + \log|H : K_{j(x)}|, \qquad 1 + \varepsilon_U = \frac{|b|\,|H|}{N} = \frac{1}{N} \sum_{h \in H} \varphi_{\langle h \rangle}(b),$$

where $\varphi_C(b) = \sum_j a_j \,|\{gK_j : C \le gK_j g^{-1}\}|$ is the mark of the cyclic subgroup C on b.

Proof. The orbits of $\rho_b(H)$ are the summands, and conjugation by σ maps orbits to orbits. The orbit of y is its summand $H/K_{j(x)}$, of size $|H : K_{j(x)}|$, and the stabiliser of the coset $h_0 K_{j(x)}$ is K_x. The excess follows from Theorem 3.5(iv) with $|U| = |H|$ (faithfulness); the mark formula is Burnside's lemma, since $|\mathrm{Fix}(\rho_b(h))| = \varphi_{\langle h \rangle}(b)$. □

Corollary 10.3 (Only shapes matter)**.** *$\mathcal{L}_{\Phi_H(b,\sigma)}(x)$ depends on (H, b, σ, x) only through $|b|$ and the size of the summand containing $\sigma^{-1}x$. Hypotheses with the same multiset $\lambda(b)$ of cell sizes and with x in a cell of the same size have equal $\mathcal{L}$, whatever the abstract groups.*

The abstract group enters $\mathcal{L}$ only through the shapes it can realise; it enters the search through the cost of the hypothesis and through the moves available below it. This is the coordinate form of the observation of [1] that a permutation partition sees only cell sizes.

Theorem 10.4 (Mackey form of $\mathcal{L}$)**.** *For $L \le H$ let $U_L = \sigma \rho_b(L) \sigma^{-1} \le U$. Then*

$$\#\mathrm{orb}(U_L) = \sum_j a_j \,|L \backslash H / K_j|, \qquad |U_L \cdot x| = |L : L \cap K_x|,$$

$$\mathcal{L}_{U_L}(x) = \log \sum_j a_j \, |L\backslash H/K_j| \;+\; \log |L : L \cap K_x|.$$

Proof. By the Mackey formula (1) each summand H/K_j splits under L into $|L\backslash H/K_j|$ orbits. The L-orbit of the coset $h_0 K_{j(x)}$ has size $|L : L \cap K_x|$ by orbit–stabiliser. □

Corollary 10.5 (Collapse and value in coordinates)**.** *Below U_L the uncosted optimum and the value bound of Theorem 8.4 are*

$$\min_{M \le L} \mathcal{L}_{U_M}(x) = \log \sum_j a_j \, |S\backslash H/K_j|, \qquad \overline{V}(U_L, x) = \log \frac{\sum_j a_j |L\backslash H/K_j| \cdot |L : S|}{\sum_j a_j |S\backslash H/K_j|},$$

with $S = L \cap K_x$, the stabiliser of x in L. For $L = H$,

$$\overline{V}(U, x) = \log \frac{|b| \, |H : K_{j(x)}|}{\sum_j a_j |K_{j(x)}\backslash H/K_j|}.$$

Proof. Theorem 4.1 applied to U_L: the minimiser is the stabiliser U_S, whose $\mathcal{L}$ is $\log \#\mathrm{orb}(U_S)$, given by Theorem 10.4 with $L = S$ (and $|S : S \cap K_x| = 1$). The bound is the difference. Double-coset counts are invariant under conjugating K_x back to $K_{j(x)}$. □

Proposition 10.6 (The lattice below a hypothesis)**.** *$L \mapsto U_L$ is an isomorphism from the subgroup lattice of H onto the lattice of subgroups of U. Consequently the down-set of a Burnside hypothesis is the subgroup lattice of its abstract group, independently of N, b and σ, and by Theorem 10.4 every value of $\mathcal{L}$ on it is determined by the coefficients a_j, the summand type $j(x)$, the coset h_0, and the table of double-coset counts $|L\backslash H/K_j|$ of H.*

Proof. ρ_b is faithful, so $\sigma\rho_b(\cdot)\sigma^{-1}$ is an isomorphism $H \to U$, which induces a lattice isomorphism on subgroups. □

This is the main computational fact of the permutation search. The lattice below a hypothesis has $|\mathrm{Sub}(H)|$ elements (30 for S_4, 156 for S_5, 1455 for S_6), it does not grow with the length n of the string, and every quantity the search needs on it is a table lookup. The string enters only through the position of $\sigma^{-1}x$ in the layout.

10.3 Moves in Burnside coordinates

A state of the permutation search is a quadruple (H, b, L, σ): an abstract group from a library, a type, a current subgroup $L \le H$ (initially H), and a placement written as a word. Its group is $U_L = \sigma\rho_b(L)\sigma^{-1}$. Table 2 lists the moves.

Sideways moves. Type moves change the isomorphism type of the H-set and keep $|U| = |H|$; by [1, Theorem 9.2(iii)] types related by $\mathrm{Aut}(H)$ give conjugate groups, so types are stored up to $\mathrm{Aut}(H)$. Relabelling by a cheap permutation γ moves the point: $\mathcal{L}_{U^\gamma}(x) = \mathcal{L}_U(\gamma^{-1}x)$. Natural alphabets on the cube are bit flips, coordinate transpositions and transvections, at $O(\log n)$ bits each; a point transposition $(u\,v)$ costs $2n$ bits and makes every placement reachable at the price that Theorem 10.1 predicts.

Down moves are cheap. Once H, b and σ are paid for, every subgroup of H costs at most $\log|\mathrm{Sub}(H)|$ more bits, by Theorem 10.6. This is the decisive difference with moves from the root of $\mathrm{Sym}(X)$ (below): Burnside coordinates enter the lattice at a cheap, structured point and explore a small, fully tabulated down-set.

Move	Effect	Direction	Code length
type move	add or remove a summand $[H/K_j]$ against the pool of fixed points, or swap two summands in the layout	sideways (isomorphic groups)	$O(\log r + \log a_j)$
relabel γ	$\sigma \mapsto \gamma\sigma$; equivalent to replacing x by $\gamma^{-1}x$	sideways (conjugate)	$\kappa(\gamma)$
restrict	$L \mapsto M$, M maximal in L; cells split by Mackey	down	$\log r(L) + \log \lvert L : N_L(M)\rvert$, at most $\log \lvert\mathrm{Sub}(H)\rvert$
interval move	$L \mapsto M$ with $S \le M \le L$; the cell of x shrinks to a block	down, $\mathcal{L}$ non-increasing	as restrict
unrestrict	$M \mapsto L$	up within $\mathrm{Sub}(H)$	as restrict
extend	$H \mapsto H' \ge H$ when $b = \mathrm{Res}^{H'}_{H} b'$	up	$O(\log)$ in a library
join π	$U \mapsto \langle U, \pi\rangle$	up, leaves coordinates	$\kappa(\pi)$

Table 2: Moves of the permutation search in Burnside coordinates. $S = L \cap K_x$ is the stabiliser of x in L.

Interval moves and blocks. By Theorem 8.3 the subgroups M with $S \le M \le L$ correspond to the blocks of L on the orbit of x, and moving to M shrinks the cell of x to a block without breaking its stabiliser, so $\mathcal{L}$ does not increase. In coordinates these are the subgroups between $L \cap K_x$ and L in $\mathrm{Sub}(H)$.

Since $|U_L \cdot x| \le |H|$, models with cells larger than $|H|$ need up moves beyond H: extension to an overgroup in the library, or joins. For example, with $H = (\mathbb{Z}/2)^d$ and $H' = (\mathbb{Z}/2)^{d+1}$, $\mathrm{Res}^{H'}_{H}[H'/1] = 2[H/1]$, so a type of regular orbits extends and adjacent cells merge.

Moves from the root. From the root of $\mathrm{Sym}(X)$ a down move to a maximal subgroup (O'Nan–Scott [14, 9]) costs about the uniform code of a set or a block system, so the root alone reaches nothing compressive (Section 6). Burnside coordinates avoid this by entering the lattice directly at (H, b, σ).

10.4 Costs, seeds and the strong sublattice in $\mathrm{Sym}(X)$

The cost of a state. A state is coded in four parts,

$$c(H, b, L, \sigma) = c(H) + c(b \mid H) + c(L \mid H) + c(\sigma),$$

with $c(H)$ an index in a library of small groups, $c(b \mid H)$ a sparse code for the coefficients (the number of non-trivial summands, their classes and multiplicities, the remaining points fixed), $c(L \mid H) \le \log|\mathrm{Sub}(H)|$, and $c(\sigma)$ the length of the placement word. By Theorem 6.8 and Theorem 6.2 every cell of a state of cost at most ε is an $(\varepsilon + O(\log n))$-strong model. Since compressive hypotheses cost less than n bits, the placement must be a short word: the strong sublattice of $\mathrm{Sym}(X)$ consists of cheap types placed by cheap relabellings.

Seeds: natural actions on the cube. The structured groups acting on $\{0,1\}^n$ are the natural seeds of the permutation search. Each is described in $O(\log n)$ bits, plus the cost of any named set.

Algorithm 9 EvaluateBurnside$(x, (H, b, L, w), \lambda)$

Require: string x; state with $b = \sum_j a_j[H/K_j]$ (sparse), $L \in \mathrm{Sub}(H)$, placement word $w = \gamma_1 \cdots \gamma_t$

$y \leftarrow x$; **for** $i = 1, \ldots, t$ **do** $y \leftarrow \gamma_i^{-1}(y)$ ▷ $y = \sigma_w^{-1} x$

locate the summand of y from the layout offsets: type j, coset $h_0 K_j$; $K_x \leftarrow h_0 K_j h_0^{-1}$

$S \leftarrow L \cap K_x$ ▷ stabiliser of x in L, a lookup in $\mathrm{Sub}(H)$

$\nu_L \leftarrow \sum_j a_j |L\backslash H/K_j|$; $\nu_S \leftarrow \sum_j a_j |S\backslash H/K_j|$ ▷ table lookups

$\mathcal{L} \leftarrow \log \nu_L + \log |L : S|$; $\overline{V} \leftarrow \mathcal{L} - \log \nu_S$ ▷ Theorems 10.4 and 10.5

$\Phi \leftarrow \lambda\, c(H, b, L, w) + \mathcal{L}$; $\Phi_{\mathrm{low}} \leftarrow \lambda\, c(H, b, L, w) + \log \nu_S$

return $\mathcal{L}, \Phi, \overline{V}, \Phi_{\mathrm{low}}$ and features

Seed	Orbit partition of X
$\mathrm{Sym}(X)$, $\mathrm{Alt}(X)$, $\mathrm{AGL}(n, 2)$, translations $T \cong \mathbb{F}_2^n$, $T \rtimes S_n$	one orbit
$\mathrm{GL}(n, 2)$	$\{0\}$ and the rest
coordinate permutations S_n	the $n + 1$ weight classes
$S_k \times S_{n-k}$ on two coordinate blocks	pairs of weights on the blocks, $(k + 1)(n - k + 1)$ cells
translations by vectors supported on $I \subseteq [n]$	subcubes with fixed coordinates outside I
Young subgroup Y_B of a named set B	$\{B, X \setminus B\}$, at extra cost $C(B)$

Example 10.7 (Weight classes)**.** For the coordinate group S_n, $\mathcal{L}_{S_n}(x) = \log(n + 1) + \log \binom{n}{|x|}$ at cost $O(\log n)$: the familiar two-part code by Hamming weight, sufficient for strings that are typical given their weight. The closed subgroup of this partition in $\mathrm{Sym}(X)$ is the Young subgroup of the weight classes, with the same orbits and cost but no trace of S_n; the permutation matrices are its closed subgroup in $\mathrm{GL}(n, 2)$ [1, Example 9.5]. A search that records descriptions keeps the symmetry that explains the partition.

The seeds are the meeting point of the two searches of this paper: $\mathrm{GL}(n, 2)$ and $\mathrm{AGL}(n, 2)$ are seeds of the permutation search, and the down-set of $\mathrm{GL}(n, 2)$ is the search space of Section 11 (Theorem 12.1).

10.5 Algorithms in Burnside coordinates

Precomputation. For each abstract group H in the library, compute once: its subgroup lattice with maximal-subgroup relations and normaliser indices, representatives K_j of the classes, the table of double-coset counts $|L\backslash H/K_j|$ for all $L \in \mathrm{Sub}(H)$ and all j, and the table of marks [21]. For the groups that matter (order up to a few hundred) these tables are small.

Algorithm 9 never builds a permutation of the $N = 2^n$ points: it applies t letters to one point, locates it in the layout, and reads tables. Its cost is $O(t\, c_\Gamma + s)$ plus a constant number of lookups, where c_Γ is the cost of a letter on a point and s the number of non-trivial summands.

Canonical keys and transpositions.

Proposition 10.8 (What the value depends on)**.** *For moves within* $\mathrm{Sub}(H)$*, with class-then-conjugate codes, the optimal value of a state* (H, b, L, σ) *at* x *depends only on* H*,* b*,* L*, the summand type* j *of* $\sigma^{-1}x$*, and the double coset* Lh_0K_j*.*

Proof. Replacing h_0 by lh_0k with $l \in L$, $k \in K_j$ replaces K_x by lK_xl^{-1}, so the $\mathcal{L}$-value of $M \leq L$ becomes that of $l^{-1}Ml$ (Theorem 10.4). Conjugation by l is an automorphism of $\mathrm{Sub}(L)$ that preserves class-then-conjugate code lengths, so the minimum over the down-set is unchanged. □

MCLS therefore keys nodes by $(H, [b], L, j, Lh_0K_j)$: positions of x in the same double coset share a node, which is the transposition structure of the permutation search.

Proposals. Candidates for a state are generated from x: interval moves from the subgroups between S and L; restrictions to maximal subgroups M of L ranked by $|M \cap K_x|$, which keeps the stabiliser; type moves that put $\sigma^{-1}x$ into a summand with a larger stabiliser; relabellings γ for which $\gamma^{-1}x$ lies in such a summand; extensions in the library; and random moves for exploration.

Features. Besides the general invariants of Section 8.4: $|b|$ and the histogram of $\log|H : K_j|$ weighted by a_j (the shape $\lambda(b)$), $j(x)$ and $|H : K_{j(x)}|$, the marks of S and L on the summands, the depth of L in $\mathrm{Sub}(H)$, and the cost split $c(H)$, $c(b \mid H)$, $c(L \mid H)$, $c(\sigma)$.

11 Linear symmetry: $\mathrm{GL}(n, 2)$

11.1 The ambient group $\mathrm{GL}(n, 2)$

Identify X with $\mathbb{F}_2^n$ and take $G = \mathrm{GL}(n, 2)$; since linear maps fix 0, $x \neq 0$ throughout. By [1, Theorem 6.3] the cells of $\mathrm{Par}_{\mathrm{GL}}$ are $\{0\}$ and the *linearly homogeneous* sets: those on which the permutations preserving all $\mathbb{F}_2$-linear dependencies act transitively. Affine subspaces are cells of cheap hypotheses, and the linear structure function lies in the band $C(x)-\alpha \leq h_x^{\mathrm{GL}}(\alpha) \leq n-\alpha$ [1, Theorem 7.1]. Both edges are attained: by strings whose best models are affine [1, Corollary 8.1] and by stochastic, normal strings for which $h_x^{\mathrm{GL}}(\alpha) \geq n - \alpha - O(t + \log n)$ up to $\alpha \approx C(x)$ [1, Theorem 8.2]. The latter have simple sufficient statistics, computable from x by short total programs, that no linear-symmetric hypothesis finds [1, Corollary 8.3]; the proof only uses that the class has $2^{\mathrm{poly}(n)}$ decidable cells.

In coordinates a subgroup is an abstract group with a faithful $\mathbb{F}_2H$-module of dimension n, placed by a matrix, and $C(H, b) \leq C(U) \leq C(H, b)+n^2$ up to $O(\log n)$ [1, Proposition 9.6]. The placement cannot hide much information, which is what makes the space searchable; it is also small, which is what makes it blind [1, Section 9.3]. Unlike $\mathrm{Sym}(X)$, closures keep structure: the closed subgroup of the weight-class partition in $\mathrm{GL}(n, 2)$ is the group of permutation matrices (Theorem 10.7).

11.2 The objective in $\mathrm{GL}(n, 2)$

For $u \in \mathrm{GL}(n, 2)$, $\mathrm{Fix}(u) = \ker(u - I)$, so

$$\varepsilon_H = \sum_{u \in H \setminus \{1\}} 2^{-\operatorname{rank}(u-I)}, \qquad \#\mathrm{orb}(H) = \frac{1}{|H|} \sum_{C \in \mathrm{ccl}(H)} |C|\, 2^{\,n-\operatorname{rank}(c_C-I)},$$

and Theorem 3.4 reads $\mathcal{L}_H(x) = n + \log\left(1 + \sum_{u \neq 1} 2^{-\operatorname{rank}(u-I)}\right) - \log|\mathrm{Stab}_H(x)|$. The top value is $\log 3$ (Theorem 4.3).

Example 11.1 ($n = 3$, linear)**.** $\mathrm{GL}(3, 2)$ has order 168 and is transitive on the 7 non-zero vectors, so $S = \mathrm{Stab}_{\mathrm{GL}}(x)$ has order 24, there are 7 such stabilisers, and $\mathrm{Fix}(S) = \{0, x\}$. Given S, $\mathcal{L} = \log 3 \approx 1.58$ bits; naming S costs $\log 7 \approx 2.81$ bits; the total, $\log 21 \approx 4.39$ bits, exceeds the 3 bits of x.

Example 11.2 (Two groups, one subspace)**.** Let $V = W \oplus W'$ with W spanned by k coordinate vectors and W' by the others, and $x \in W \setminus \{0\}$.

1. $H_1 = \mathrm{GL}(W) \times 1_{W'}$: orbits $\{w'\}$ and $w' + (W \setminus \{0\})$ for $w' \in W'$, so $\#\mathrm{orb}(H_1) = 2^{n-k+1}$, $|H_1 \cdot x| = 2^k - 1$ and $\mathcal{L}_{H_1}(x) \approx n + 1$. The cell $W \setminus \{0\}$ is special (the one with $w' = 0$) and costs $O(1)$ bits to name given the partition, but the uniform index charges $n - k + 1$.

2. $H_2 = 1_W \times \mathrm{GL}(W')$: x is fixed, the orbits are $\{w\}$ and $w + (W' \setminus \{0\})$, so $\#\mathrm{orb}(H_2) = 2^{k+1}$ and $\mathcal{L}_{H_2}(x) = k + 1$.

Both cost $O(\log n)$. Under $\mathcal{L}$ the subspace structure of x is seen by H_2 and missed by H_1. With the fixed-set code of Section 4.3, which in $\mathrm{GL}(n, 2)$ names x by its coordinates in the subspace $\mathrm{Fix}(H)$ at cost $\dim \mathrm{Fix}(H)$, H_2 costs exactly k bits for the cell; and the planted-string gap $\log \#\mathrm{orb}(P) - \log |\mathrm{Fix}(P)|$ of Theorem 8.6 is one bit for $P = H_2$.

11.3 Moves, seeds and the strong sublattice in $\mathrm{GL}(n, 2)$

From the root. The maximal subgroups of $\mathrm{GL}(n, 2)$ are classified by Aschbacher's theorem [3, 12]; under the class-then-conjugate code a k-dimensional subspace costs about $k(n-k)$ bits, and every first step from the root costs at least about n bits (Section 6). Structured placements need seeds.

Seeds. Structured subgroups with $O(\log n)$ to $O(k \log n)$ bit descriptions:

Seed	Orbits on $\mathbb{F}_2^n$
block-diagonal $\mathrm{GL}(W_1) \times \cdots \times \mathrm{GL}(W_k)$ on coordinate intervals	products of $\{0\}$ and $W_i \setminus \{0\}$
pointwise stabiliser $1_W \times \mathrm{GL}(W')$ of a coordinate subspace	$\{w\}$, $w + (W' \setminus \{0\})$ (Theorem 11.2)
permutation matrices S_n	weight classes
upper unitriangular group	sets of vectors with the same last non-zero coordinate
Singer cycle $\mathbb{F}_{2^n}^\times$	$\{0\}$ and the rest; subgroups give cosets of multiplicative subgroups
$\mathrm{Sp}(n, 2)$ for the standard form on coordinate pairs (n even)	$\{0\}$ and the rest; subgroups refine by the form

Restriction to maximal subgroups and relabelling by transvections $I + E_{ij}$ and coordinate transpositions, each $O(\log n)$ bits, move from the seeds. By Theorem 6.8 every hypothesis so reached within budget ε gives $(\varepsilon + O(\log n))$-strong models.

Placement. A placement costs at most n^2 bits, but compressive hypotheses cost less than n, so only placements with short words matter. For example, every affine subspace $A \ni x$ is a cell of a hypothesis whose type is cheap, but its placement costs $O(n^2)$ bits unless A is structured; the affine models that the search can find within a budget below n are the structured ones.

11.4 Algorithms and validation in $\mathrm{GL}(n, 2)$

Evaluation. Algorithm 1 with $|\mathrm{Fix}(c)| = 2^{n - \mathrm{rank}(c-I)}$; point stabilisers through a faithful permutation representation (Schreier–Sims); $\mathrm{Fix}(H)$ as the null space of the stacked matrix $[u_1 - I; \ldots; u_g - I]$ over generators. Orbits on $\mathbb{F}_2^n$ are never enumerated.

Proposals. Interval moves from blocks of H on $H \cdot x$ (when the orbit is small) or maximal subgroups containing $\mathrm{Stab}_H(x)$; stabilisers of subspaces built from x, such as $\langle x \rangle$, coordinate subspaces containing x and $\mathrm{Fix}(\mathrm{Stab}_H(x))$, when they are expressible in the scheme; relabellings by transvections that move x into larger fixed spaces.

Module-theoretic features. In addition to the general invariants: $\dim \mathrm{Fix}(H)$ and $\dim \mathrm{Fix}(\mathrm{Stab}_H(x))$; the dimension of the cyclic submodule generated by x, which is the span of $H \cdot x$, and of its radical and socle layers; and the composition factors of the module. All are GL-invariants of the pair (H, x).

What to expect. By [1, Corollary 8.1] there are strings of every admissible profile whose best models are affine, and strings planted in fixed spaces of cheap groups have near-sufficient reachable models (Theorem 8.6): the search has targets. Any budget $\varepsilon = \mathrm{poly}(n)$ gives a class of $2^{\mathrm{poly}(n)}$ decidable cells, so by the maximal gap theorem there are stochastic, normal strings on which every model in the class is nearly useless: a small root value means "no linear structure found", not "x is random". And a search that stalls inside a refinement interval $[\mathrm{Stab}_H(x), H]$ with no cheaply described block of about half the size (Theorem 8.3) may have found a plateau of h_x^{GL}, which the block structure of H on $H \cdot x$ distinguishes from a local optimum of the search.

12 The two searches together

Proposition 12.1 (Nesting)**.** $\mathrm{GL}(n,2) \le \mathrm{AGL}(n,2) \le \mathrm{Sym}(X)$. *Every $H \le \mathrm{GL}(n,2)$, viewed as a permutation group on X, has the same orbits and the same $\mathcal{L}_H(x)$. If the permutation search has $\mathrm{GL}(n,2)$ as a seed with an $O(1)$ code and uses the same move codes below it, then $c_{\mathrm{Sym}}(H) \le c_{\mathrm{GL}}(H) + O(1)$ for every hypothesis of the linear search. Consequently every profile point found by the linear search is available to the permutation search at an additive $O(1)$ cost, and up to $O(\log n)$, $h^{\mathrm{Sym}}_{x,\varepsilon} \le h^{\mathrm{GL}}_{x,\varepsilon}$ and $h^{\mathrm{Sym}}_x \le h^{\mathrm{GL}}_x$ [1, (6.1) and Proposition 4.2].*

Proof. The inclusions are standard ($\mathrm{GL}(n,2)$ is the stabiliser of 0 in $\mathrm{AGL}(n,2)$). $\mathcal{L}$ depends only on orbits. A linear word is a permutation word after an $O(1)$ prefix naming the seed. □

The linear search is thus the down-set of one seed of the permutation search, and the point of running both is the difference between them. Running MCLS from the seeds of each search on the same string gives two archives of profile points and two best models. If both find a near-sufficient cheap model, x has linear-symmetric structure; if only the permutation search does, x has simple structure that is not linear-symmetric, as for the blind strings of [1, Theorem 8.2], and the cost split of the permutation model shows whether that structure is a symmetry with an informative type or a named set. Empirical sophistications computed from the two archives estimate the symmetry gap $\mathrm{soph}_{\mathrm{GL}}(x) - \mathrm{soph}(x)$ recommended in [1, Section 9.3(a)]; both are estimates from above, so the difference is reported with both values and both models, not as a bound. The table summarises the two searches.

	$\mathrm{Sym}(X)$, Burnside coordinates	$\mathrm{GL}(n,2)$
cells	all non-empty sets	$\{0\}$ and linearly homogeneous sets
type	Burnside class $b \in B(H)$	faithful $\mathbb{F}_2 H$-module
$\mathcal{L}$ in coordinates	$\log \lvert b \rvert + \log \lvert H : K_{j(x)} \rvert$	$n + \log(1 + \sum_{u \ne 1} 2^{-\mathrm{rank}(u-I)}) - \log \lvert \mathrm{Stab}_H(x) \rvert$
uncosted optimum	1 bit	$\log 3$ bits
value bound $\overline{V}$	ratio of double-coset counts	$\delta_H - \delta_{\mathrm{Stab}_H(x)}$ from ranks
lattice below a hypothesis	$\mathrm{Sub}(H)$, tabulated	subgroups of a matrix group
placement cost	up to about $n2^n$ bits; dominates	at most n^2 bits
first step from the root	at least n bits (except Alt)	at least $\log(2^n - 1)$ bits
natural seeds	actions on the cube; named sets	coordinate block groups, permutation matrices, forms
completeness	every strong model (in principle)	linearly homogeneous models only
blindness	none for the class; the language limits it	maximal gap theorem

Outlook. This paper sets up model search on subgroup lattices as value estimation and gives the theory and the algorithms; it reports no experiments. The next step is an implementation

of MCLS with the networks of Section 9.5 for both ambient groups, validated against exact values on small lattices (Section 9.8) and on planted strings, and then run at sizes where the lattice can no longer be enumerated.

References

[1] R. Banerjee. CAS II: Orbits as Models. Kolmogorov's Structure Function under Symmetry. Manuscript, October 2026. `https://arxiv.org/pdf/2609.40290`.

[2] M. Andrychowicz et al. Hindsight experience replay. In *Advances in Neural Information Processing Systems 30*, 2017.

[3] M. Aschbacher. On the maximal subgroups of the finite classical groups. *Invent. Math.* 76:469–514, 1984.

[4] D. P. Bertsekas and J. N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

[5] C. B. Browne et al. A survey of Monte Carlo tree search methods. *IEEE Trans. Comput. Intell. AI Games* 4(1):1–43, 2012.

[6] P. J. Cameron, R. Solomon and A. Turull. Chains of subgroups in symmetric groups. *J. Algebra* 127 (1989), 340–352.

[7] B. E. Childs, J. H. Brodeur and L. Kocsis. Transpositions and move groups in Monte Carlo tree search. In *IEEE Symp. Computational Intelligence and Games*, 2008.

[8] A. Couëtoux, J.-B. Hoock, N. Sokolovska, O. Teytaud and N. Bonnard. Continuous upper confidence trees. In *LION*, 2011.

[9] J. D. Dixon and B. Mortimer. *Permutation Groups*. Graduate Texts in Mathematics 163, Springer, 1996.

[10] A. Fawzi et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature* 610 (2022), 47–53.

[11] P. D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.

[12] P. Kleidman and M. Liebeck. *The Subgroup Structure of the Finite Classical Groups*. LMS Lecture Note Series 129, Cambridge University Press, 1990.

[13] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In *ECML*, 2006.

[14] M. W. Liebeck, C. E. Praeger and J. Saxl. A classification of the maximal subgroups of the finite alternating and symmetric groups. *J. Algebra* 111:365–383, 1987.

[15] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi and Y. W. Teh. Set Transformer: a framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[16] L.-J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning* 8:293–321, 1992.

[17] A. Milovanov. Algorithmic statistics: normal objects and universal models. In *CSR 2016*, LNCS 9691, Springer, 2016.

[18] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature* 518:529–533, 2015.

[19] A. Y. Ng, D. Harada and S. Russell. Policy invariance under reward transformations: theory and application to reward shaping. In *ICML*, 1999.

[20] OEIS Foundation. The On-Line Encyclopedia of Integer Sequences, sequence A005432 (number of subgroups of S_n). `https://oeis.org/A005432`.

[21] G. Pfeiffer. The subgroups of M_{24}, or how to compute the table of marks of a finite group. *Experiment. Math.* 6(3):247–270, 1997.

[22] L. Pyber. Asymptotic results for permutation groups. In *Groups and Computation*, DIMACS Ser. Discrete Math. Theoret. Comput. Sci. 11, AMS, 1993, 197–219.

[23] J. Rissanen. Modeling by shortest data description. *Automatica* 14(5):465–471, 1978.

[24] C. M. Roney-Dougal and G. Tracey. Subgroups of symmetric groups: enumeration and asymptotic properties. arXiv:2503.05416, 2025.

[25] T. Schaul, D. Horgan, K. Gregor and D. Silver. Universal value function approximators. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015.

[26] A. Saffidine, T. Cazenave and J. Méhat. UCD: Upper confidence bound for rooted directed acyclic graphs. *Knowledge-Based Systems* 34:26–33, 2012.

[27] M. P. D. Schadd, M. H. M. Winands, H. J. van den Herik, G. M. J.-B. Chaslot and J. W. H. M. Uiterwijk. Single-player Monte-Carlo tree search. In *Computers and Games*, LNCS 5131, Springer, 2008.

[28] D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature* 529 (2016), 484–489.

[29] D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* 362:1140–1144, 2018.

[30] L. Solomon. The Burnside algebra of a finite group. *J. Combinatorial Theory* 2:603–615, 1967.

[31] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[32] J. Tromp. The number of legal Go positions. In *Computers and Games (CG 2016)*, LNCS 10068, Springer, 2016, 183–190.

[33] N. Vereshchagin. Algorithmic minimal sufficient statistics: a new approach. *Theory Comput. Syst.* 56(2), 2015.

[34] N. K. Vereshchagin and P. M. B. Vitányi. Kolmogorov's structure functions and model selection. *IEEE Trans. Inform. Theory* 50(12):3265–3290, 2004.